Build a Unix Shell in C

Implementation guide for **mysh**: a small, tested, memory-clean shell for Ubuntu/Linux

Scope	Interactive shell with quoting, redirection, pipelines, built-ins (cd, pwd, exit) and Ctrl-C handling. About 400-600 lines of C plus unit and integration tests.
Time budget	About 32 focused hours: 7-10 days at 2-6 h/day. Core only; extensions are optional and gated.
Outcome	A finished repository you can demo and defend in a technical interview, then a clean switch to timed practice.
Platform	Ubuntu/Linux, GCC, GNU Make, GDB, Git. C11 plus POSIX.1-2008 interfaces.

How this guide was checked. The build commands, Makefile, test scripts and code fragments were exercised against a throw-away reference build (Ubuntu 24.04, GCC 13.3, GNU Make 4.3). Deliberate bugs (an unclosed pipe end, a missing `waitpid`, a leaked descriptor) were injected to confirm that the tests described here catch them. The reference is intentionally **not** included: you write the shell. Outputs shown from sanitizers and compilers are real but trimmed; addresses and paths will differ on your machine.

How to use this guide

Keep it open next to your editor. Every implementation stage has the same shape, so you can jump straight to what you need:

Section	What it gives you
Objective, estimate, needs	What you build, how long it should take, which earlier stages it depends on.
Concepts	Only the C / POSIX ideas this stage needs, explained where they first matter.
Tasks	An ordered work list with time boxes. Do them in order; commit after each.
Interfaces	Function cards: signature, responsibility, parameters, modifies, memory ownership, errors, callers.
Examples and pitfalls	Focused fragments and pseudocode (never a full solution), with flawed-vs-correct comparisons.
Testing	Concrete test cases with expected results and the tools to investigate failures.
Completion criteria	Observable results. If every row is true, the stage is finished: move on.
Checkpoint questions	Questions to work out yourself, with a general hint only. They train the reasoning interviewers probe.

Code conventions. A code block captioned *complete* compiles as shown. A block captioned *fragment* or *pseudocode* is deliberately incomplete: you supply the rest. Fragments use the exact names and signatures of the final design, so everything fits together.

> **NOTE — time-boxing**
> If you get stuck for more than about 30 minutes on one bug, stop and apply the debugging routine in Chapter 3 instead of staring at the code. Then, if still stuck, ask for help with a minimal failing test case in hand.

Contents

1. Project overview and design 4

1.1 What mysh is 4

1.2 Scope decisions 4

1.3 Architecture 5

1.4 Command lifecycle and data flow 6

1.5 Project-wide conventions 6

2. Roadmap and time plan 7

2.1 Day-by-day plan 7

2.2 Scope-cut ladder 8

Step 0. Environment and workflow setup 9

0.1 Install and verify the toolchain 9

0.2 Create the repository and layout 9

0.3 Compile a program and understand every part of the command 9

0.4 The Makefile 11

0.5 Git workflow 12

0.6 First commit: a stub that builds 12

3. Debugging and testing toolkit 14

3.1 Which tool for which symptom 14

3.2 A debugging routine that works . . . 14
3.3 Reading sanitizer reports . . . 14
3.4 GDB essentials . . . 15
3.5 strace and valgrind (optional, install separately) . . . 16
3.6 Extra compiler checks worth one pass in Stage 9 . . . 16
3.7 Test strategy at a glance . . . 16

Stage 1. Skeleton, memory helpers and line input . . . 17

Stage 2. The lexer: from characters to tokens . . . 21

Stage 3. Command representation and the parser . . . 26

Stage 4. Running external commands . . . 30

Stage 5. Built-in commands and shell state . . . 36

Stage 6. Redirection and file descriptors . . . 39

Stage 7. Pipelines . . . 43

Stage 8. Signals and robustness . . . 47

Stage 9. Hardening and test consolidation . . . 51

9.1 Ownership audit . . . 51
9.2 Error-path audit . . . 51
9.3 Parser fuzz test . . . 52
9.4 Stress and resource checks . . . 52

Stage 10. Documentation, repository and CV packaging . . . 54

11. Optional extensions and the stopping rule . . . 57

11.1 When you may take an extension . . . 57
11.2 Not planned, and why . . . 57
11.3 The stopping rule . . . 57
11.4 Switching to timed practice . . . 57

12. Final project completion checklist . . . 58

13. References and further reading . . . 59

Appendix A. Complete public interface (all headers) . . . 60

1. Project overview and design

1.1 What mysh is

`mysh` reads a line, splits it into words and operators, builds a small command structure, and runs it with the same system calls a real shell uses: `fork`, `execvp`, `waitpid`, `pipe`, `open`, `dup2`. It works interactively (prompt, Ctrl-C, Ctrl-D) and non-interactively (a script on stdin), which is also how the automated tests drive it.

The scope is chosen so that every feature teaches something specific about C or Unix and earns its cost. It is *not* an attempt at Bash.

mysh grammar (informal)

```
line      := [ pipeline ]                      empty line = do nothing
pipeline  := command { "|" command }
command   := { word | redirect }               at least one word is required
redirect  := ( "<" | ">" | ">>" ) word
word      := unquoted characters, 'single quoted', "double quoted" and \x escapes,
             concatenated:  ab"cd"'ef'  is the single word  abcdef
```

Topic	Rule
Quoting	`'...'` is literal. `"..."` is literal except `\"` and `\\`. Outside quotes `\x` means the character x. Operators inside quotes are ordinary text: `echo ">"` prints `>`.
Operators	`| < > >>` are recognised without surrounding spaces (`ls|wc`). Longest match wins: `>>>` is `>>` then `>`.
Built-ins	`cd [dir]`, `pwd`, `exit [n]`. Everything else is searched for in PATH with `execvp`.
Exit status	0 success; 1 failure of a built-in, redirection or fork; 2 syntax error; 126 found but not executable; 127 not found; 128+n killed by signal n (130 for Ctrl-C). A pipeline returns the status of its last command.
Modes	Prompt `mysh$ ` goes to stderr and appears only when stdin is a terminal. At end of input the shell exits with the last command's status.
Errors	Messages look like `mysh: <what>: <reason>` on stderr. A bad line never terminates the shell.

1.2 Scope decisions

Each candidate feature was judged on learning value, systems relevance, CV value and total cost (implement + test + debug). Pipelines and redirection are included because they are what makes a shell a shell and they teach file descriptors properly; background jobs and expansion are not, because their cost is out of proportion to what they add.

Feature	Learning / CV value	Cost	Decision
Run external commands (fork/exec/wait)	Very high	Medium	**Core**
Quoting and operator tokenizer	High: string scanning, state machines, dynamic arrays	Medium	**Core**
Parser into command structures	High: nested ownership, error paths	Medium	**Core**
Built-ins cd, pwd, exit	Medium: why built-ins must exist	Low	**Core**
Redirection `< > >>`	Very high: descriptors, dup2	Medium	**Core**
Pipelines `a | b | c`	Very high: pipe, concurrency, closing discipline	Medium-high	**Core**
Ctrl-C handling, EINTR	High: signals, async-signal-safety	Low-medium	**Core** (minimal)
Unit + integration tests, sanitizer runs	Very high for credibility	Medium	**Core**
`2>`, `2>&1` (fd-numbered redirection)	High	Low-medium	Extension E1
`$?` and `$VAR` expansion	Medium: lexer interaction with quotes	Medium	Extension E2
`; && ||` command lists	Medium: grammar grows	Medium	Extension E3
Background jobs `&`	Medium: reaping, job table	Medium-high	Extension E4
Globbing, history, line editing, scripting, job control	Low per hour spent	High	Out of scope

1.3 Architecture

Six small modules, one direction of dependency, and one rule that makes the whole design easy to hold in your head: *data flows one way, from text to tokens to a command structure to an exit status*, and each stage is independently testable. The lexer and parser never touch the operating system; only the executor and built-ins do.

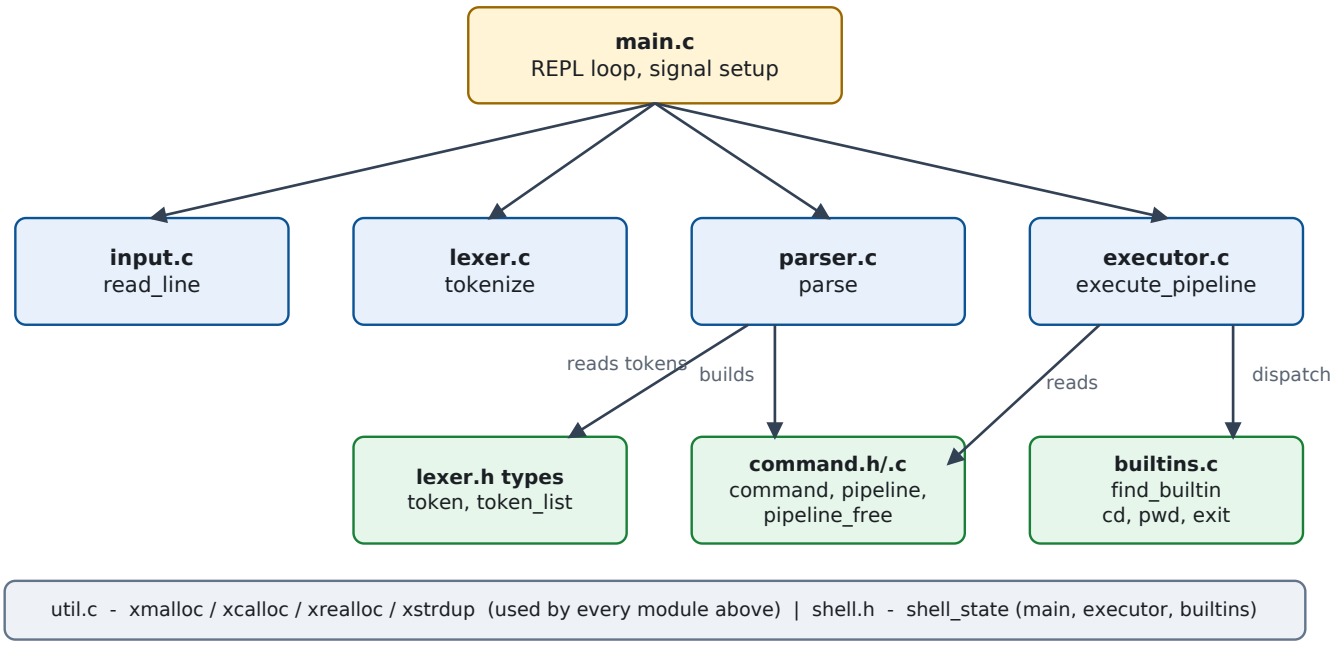


Figure 1. Modules and dependencies. Arrows point from user to used. Green boxes are data definitions and the built-in table; blue boxes are behaviour.

File	Responsibility	Depends on	Owns (allocates/frees)
`main.c`	Read-eval loop, signal setup, glue	all	The line, token list and pipeline of the current iteration
`input.c/.h`	Read one line of any length	util	Allocates the line; caller frees
`lexer.c/.h`	Characters to tokens; quoting; operators	util	Allocates token texts and the array; `token_list_free` frees
`parser.c/.h`	Tokens to `pipeline`; syntax errors	lexer, command, util	Allocates the whole pipeline (deep copy); `pipeline_free` frees
`command.c/.h`	Data types and `pipeline_free`	none	Defines who frees what
`executor.c/.h`	fork/exec, pipes, redirection, waiting	command, builtins, util	Temporary pid array; no long-lived memory
`builtins.c/.h`	cd, pwd, exit and the lookup table	shell.h	Nothing
`util.c/.h`	Abort-on-failure allocation wrappers	none	Nothing (callers own results)
`shell.h`	`shell_state`: last status, exit flag	none	Nothing

DESIGN DECISION

Why not one big file or a deeper hierarchy? One file hides the data flow and makes unit testing impossible. A deeper hierarchy (abstract interfaces, vtables, a generic container library) would add code you must then explain without teaching anything new. Eight small `.c` files is the point where each file has one job and you can still read the whole program in an afternoon.

1.4 Command lifecycle and data flow

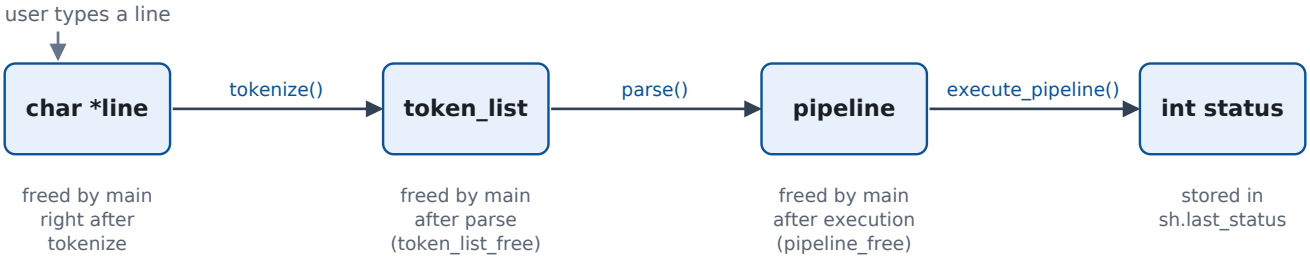


Figure 2. What each stage hands to the next, and who frees it.

1. `main` calls `read_line` and receives a heap-allocated line (or EOF / interrupted).
2. `tokenize` copies words into new strings inside a `token_list`. After it returns, the line is no longer needed and is freed.
3. `parse` validates syntax and deep-copies the words into a `pipeline` of `command`s with NULL-terminated `argv` arrays. After it returns, the tokens are freed.
4. `execute_pipeline` reads the pipeline (it never modifies or frees it). A single built-in runs in the shell process; everything else is `fork` + `execvp` per command, wired with pipes and redirections, then `waitpid`.
5. `main` stores the returned status in `shell_state.last_status`, frees the pipeline, and loops.

1.5 Project-wide conventions

These rules are applied in every stage. They are the reason the interfaces fit together; when you wonder how a function should report an error or who frees something, the answer is here.

Topic	Rule
Allocation	Use `xmalloc`, `xcalloc`, `xrealloc`, `xstrdup`. They print a message and `exit(1)` if memory runs out, so callers never check for NULL. Trade-off: simpler code, but no recovery from out-of-memory (acceptable for a shell; discussed in Stage 1).
Ownership	Whoever receives heap memory from a function owns it and must free it, unless the function card says ownership is borrowed. Borrowed pointers (`const char *`, `const pipeline *`) must not be freed or stored beyond the call.
`*_free` functions	Free everything the structure owns, reset it to the empty state, and are safe to call twice. They never free the structure itself (callers keep it on the stack).
User-input errors	Lexer and parser return `0` or `-1` and set `*err` to a static string literal (never freed). They do not print; `main` prints `mysh: syntax error: <msg>`.
Failure state	After a failed call the out-parameter is in the empty state, so the caller can free it unconditionally.
System-call errors	Print `mysh: <context>: <strerror(errno)>` to stderr, return/exit with the status from 1.2. Capture `errno` immediately; the next call may overwrite it.
Streams	stdout carries program output only. Prompt and diagnostics go to stderr.
Visibility	Helpers are `static`. Only functions declared in a header are public.

2. Roadmap and time plan

Estimates include writing the code, writing its tests, debugging, and committing. They assume you read the stage first and work in order. The total is the planning figure of about 32 hours.

Stage	Objective	Key concepts	Needs	Done when...	Type	Hours
0 Environment	Toolchain, repo, Makefile, first build	Compile vs link, headers, flags, sanitizers, Git	-	Stub builds with zero warnings	Core	1.5
1 Input	Memory helpers; REPL; read any line length	Pointers, strings, malloc/realloc/free, fgets, isatty	0	Echo loop handles long lines and EOF, ASan clean	Core	2.5
2 Lexer	Line to tokens with quotes and operators	State machine, dynamic array, ownership, static helpers	1	Lexer unit tests pass	Core	4.0
3 Parser	Tokens to pipeline structures; syntax errors	Nested structs, count-then-allocate, deep copy, cleanup paths	2	Parser unit tests pass; bad syntax rejected	Core	3.5
4 External commands	Run one program, report status	fork, exec family, waitpid, status macros, _exit, zombies	3	Commands run; 126/127/status correct	Core	3.5
5 Built-ins	cd, pwd, exit; shell state	Why built-ins exist, function pointers, strtol	4	cd changes the shell; exit codes right	Core	2.0
6 Redirection	`< > >>` for commands and built-ins	File descriptors, open flags, dup2, stdio buffering	5	Files created/appended/read; no fd leaks	Core	3.5
7 Pipelines	N-stage pipelines	pipe, concurrency, EOF rules, closing discipline, SIGPIPE	6	Multi-stage pipes work; no hangs/zombies/leaks	Core	4.5
8 Signals	Ctrl-C and robustness	sigaction, handlers, EINTR, async-signal-safety	7	Ctrl-C kills child, not shell	Core	2.5
9 Hardening	Audit memory/fds, stress, fuzz, regression	Ownership audit, sanitizers, error paths	8	Whole suite green under ASan/UBSan and in release	Core	2.5
10 Release	README, repo hygiene, CV material	Documentation, honest claims	9	Fresh clone builds and passes tests	Core	2.0
Extensions	E1 `2>/2>&1`, E2 expansion, E3 lists, E4 `&`	See Chapter 11	10	Only if gates in Chapter 11 are met	Optional	2-4 each
Total (core)						32.0

2.1 Day-by-day plan

Planned at roughly 4 hours a day over 8 days, leaving days 9-10 as unplanned buffer. If some days only give you 2 hours, the arithmetic is simple: 32 hours over 10 days needs an average of 3.2 hours a day, so a 2-hour day must be paid back by a 5-hour day. Do not plan to extend the deadline: use the scope-cut ladder below instead.

Day	Work	Hours	Gate at end of day
1	Stage 0 + Stage 1	4.0	`make` is warning-free; echo loop passes the Stage 1 tests
2	Stage 2 (lexer)	4.0	Lexer unit tests green under ASan
3	Stage 3 (parser)	3.5	**Gate A:** lexer and parser unit tests green
4	Stage 4 (external commands)	3.5	Integration harness exists; basic commands pass

Day	Work	Hours	Gate at end of day
5	Stage 5 + first half of Stage 6	4.0	`cd` works; `>` works for external commands
6	Rest of Stage 6 + first half of Stage 7	4.0	**Gate B:** commands, built-ins, redirection fully tested
7	Rest of Stage 7 + Stage 8	4.5	**Gate C:** pipelines and Ctrl-C work; no hangs
8	Stage 9 + Stage 10	4.5	**Release:** tag `v1.0`, then stop
9-10	Buffer only. Optional extension if every gate was met on time	-	Then start timed practice

2.2 Scope-cut ladder

If you are more than a day behind at any gate, cut in this order. Each cut is documented in the README as a known limitation rather than hidden.

1. Stage 8: keep only the SIGINT handler and the `waitpid` retry loop; skip the pty test (saves about 1 hour).
2. Stage 6: do not support redirection on built-ins run in the parent; print an error instead (saves about 1 hour).
3. Stage 2: drop backslash escapes and keep single and double quotes (saves about 30 minutes).
4. Stage 9: skip the fuzz test and the stress scripts; keep the ownership and fd audits (saves about 1 hour).

> **PITFALL**
> **Never cut:** lexer and parser unit tests, the integration harness, sanitizer runs, pipelines, the README. Those are what make the project credible. A shell without pipelines is a command launcher.

Step 0. Environment and workflow setup

Core	**Estimate:** 1.5 hours	**Needs:** nothing

0.1 Install and verify the toolchain

shell: install and check (Ubuntu)

```
sudo apt update
sudo apt install build-essential gdb git make
gcc --version        # expect GCC 11 or newer
make --version
gdb --version
git --version
```

`build-essential` provides GCC, `make` and the C library headers. Optional but useful later: `valgrind` (second opinion on leaks), `strace` (shows system calls), `man-db` and `manpages-dev` (manual pages for system calls, usually preinstalled). The POSIX manual pages (`manpages-posix-dev`) may not be available in your repositories; the online references in Chapter 13 cover the same ground.

> **NOTE — restricted environments**
> If you use WSL or a container, LeakSanitizer sometimes refuses to run because the environment restricts `ptrace`. Symptom: a message that LeakSanitizer does not work under ptrace. Workarounds: run on native Linux or a VM, or use `valgrind --leak-check=full` on a build without sanitizers (Chapter 3).

0.2 Create the repository and layout

shell

```
mkdir mysh && cd mysh
git init
mkdir src tests
printf 'build/\nmysh\n' > .gitignore
```

target layout

```
mysh/
  Makefile
  README.md            written in Stage 10, stub it now
  .gitignore
  src/                 all production code, headers next to their .c files
    main.c  input.c/.h  lexer.c/.h  parser.c/.h  command.c/.h
    executor.c/.h  builtins.c/.h  util.c/.h  shell.h
  tests/
    check.h            tiny assertion macros (Stage 2)
    test_lexer.c  test_parser.c        unit tests (one program each)
    run_integration.sh                 end-to-end tests (Stage 4 onward)
  build/               object files and test binaries (git-ignored)
```

Headers live beside their sources so the build needs one include directory and each module is a visible `foo.c` + `foo.h` pair. Build output goes in `build/` so `git status` stays clean and `make clean` is a safe `rm -rf`.

0.3 Compile a program and understand every part of the command

main.c (complete)

```
#include <stdio.h>

int main(void) {
    puts("hello from C");
    return 0;
}
```

shell

```
gcc -Wall -Wextra -Wpedantic -g -fsanitize=address,undefined main.c -o myshell
./myshell
```

Part	Meaning
`gcc`	The compiler *driver*. It runs four tools in sequence: the preprocessor (expands `#include` and macros), the compiler proper (C to assembly), the assembler (to an object file) and the linker (combines object files and libraries into an executable).
`-Wall -Wextra`	Turn on large sets of useful warnings (unused variables, missing returns, suspicious comparisons...). Despite the name, `-Wall` is not "all". Treat every warning as a bug to fix, not noise.
`-Wpedantic`	Warn about constructs that are not standard ISO C (given a `-std=` option).
`-g`	Include debug information so GDB and the sanitizers can show file names and line numbers.
`-fsanitize=address,undefined`	Instrument the program with AddressSanitizer (invalid memory access, and leaks at exit on Linux) and UndefinedBehaviorSanitizer (signed overflow, bad shifts, null dereferences...). Costs about 2x speed; use it for all development and testing. Must also be given when linking so the runtime libraries are linked in.
`main.c`	The source file: one *translation unit*.
`-o myshell`	Name of the output executable. Without it, GCC writes `a.out`.
`./myshell`	Run it. The `./` is needed because the current directory is not on `PATH`.

Several source files, and what headers are for

shell

```
gcc -Wall -Wextra -Wpedantic -g -fsanitize=address,undefined \
    main.c parser.c shell.c -o myshell
```

GCC compiles each `.c` file on its own into an object file, then links the objects. Only `.c` files are listed. A header such as `parser.h` is pulled in textually by the preprocessor wherever a source file says `#include "parser.h"`; it contains *declarations* (types, function prototypes, macros) that tell the compiler what other files provide. The *definitions* (function bodies, global variables) live in exactly one `.c` file. Headers use include guards so a second inclusion in the same file is harmless:

src/parser.h (fragment)

```
#ifndef PARSER_H
#define PARSER_H
/* declarations only */
int parse(const token_list *tl, pipeline *out, const char **err);
#endif
```

> **PITFALL**
> Passing a header to GCC (`gcc parser.h ...`) does not do what you want: GCC treats it as a request to build a *precompiled header* and writes a `parser.h.gch` file. Headers are found through `#include`, never listed on the command line. Use `-Isrc` if a file outside `src/` (such as a test) needs to include them.

The same build in separate steps

shell: compile (-c) each file, then link

```
FLAGS="-std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Wpedantic -g"
gcc $FLAGS -c src/lexer.c -o build/lexer.o
gcc $FLAGS -c src/main.c  -o build/main.o
gcc build/main.o build/lexer.o -o mysh -fsanitize=address,undefined
```

`-c` stops after producing an object file. The last command only links, so it needs no warning flags but does need `-fsanitize` (when compiling with sanitizers, link with them). A Makefile automates exactly this and rebuilds only what changed.

Standard C, POSIX and Linux: why two extra flags appear

`fork`, `execvp`, `sigaction`, `strdup`, `open` and `dup2` are *POSIX*, not ISO C. If you compile with a strict `-std=c11`, glibc hides them unless you request POSIX explicitly. With GCC 13, `struct sigaction` then fails with "storage size of sa isn't known", and `strdup` triggers an implicit-declaration warning. The fix used throughout this project:

flags: ISO C11 plus POSIX.1-2008 declarations

```
-std=c11 -D_POSIX_C_SOURCE=200809L
```

(`-std=gnu11` would also work, but enables GNU extensions silently; the explicit macro lets you see which interfaces are POSIX.) A few things in this guide are *Linux-specific* and are flagged when they appear: the `/proc` filesystem used for descriptor checks, the typical 64 KiB pipe capacity, and sanitizer behaviour.

Reading build errors: three different kinds

Message	Meaning	Typical cause
`error: expected ';'` or `unknown type name`	Compile error: the code is not valid C here.	Typo, missing header include, wrong order of declarations.
`warning: implicit declaration of function`	Called something the compiler has not seen declared.	Missing `#include` or missing `_POSIX_C_SOURCE`. Never ignore it.
`undefined reference to 'helper'`	Link error: declared, but no linked object defines it.	File missing from the build, typo in the name, definition missing.
`multiple definition of 'x'`	Link error: two objects define the same function or variable.	A function body or global placed in a header included from two files.

0.4 The Makefile

Type this file yourself and read each line; it is the only build tooling in the project. Test targets refer to files you create later, which is fine: they match nothing until then.

Makefile (complete; indented recipe lines must start with a TAB character)

```
CC      := gcc
STD     := -std=c11 -D_POSIX_C_SOURCE=200809L
WARN    := -Wall -Wextra -Wpedantic
OPT     ?= -O0 -g
SAN     ?= -fsanitize=address,undefined
CFLAGS  := $(STD) $(WARN) $(OPT) $(SAN) -fno-omit-frame-pointer -MMD -MP
LDFLAGS := $(SAN)

BUILD    := build
TARGET   := mysh
SRCS     := $(wildcard src/*.c)
OBJS     := $(SRCS:src/%.c=$(BUILD)/%.o)
LIB_OBJS := $(filter-out $(BUILD)/main.o,$(OBJS))
TESTS    := $(patsubst tests/%.c,$(BUILD)/%,$(wildcard tests/test_*.c))

.PHONY: all test test-unit test-integration clean

all: $(TARGET)

$(TARGET): $(OBJS)
	$(CC) $(OBJS) -o $@ $(LDFLAGS)

$(BUILD)/%.o: src/%.c | $(BUILD)
	$(CC) $(CFLAGS) -c $< -o $@

$(BUILD)/test_%: tests/test_%.c tests/check.h $(LIB_OBJS) | $(BUILD)
	$(CC) $(CFLAGS) -Isrc tests/test_$*.c $(LIB_OBJS) -o $@ $(LDFLAGS)

$(BUILD):
	mkdir -p $(BUILD)

test-unit: $(TESTS)
	@for t in $(TESTS); do echo "== $$t"; ./$$t || exit 1; done

test-integration: $(TARGET)
	./tests/run_integration.sh

test: test-unit test-integration

clean:
	rm -rf $(BUILD) $(TARGET)

-include $(OBJS:.o=.d)
```

Construct	What it does
`:=` and `?=`	`:=` assigns once. `?=` assigns only if the variable is not already set, so you can override on the command line (below).

Construct	What it does
`$(wildcard ...)`, `$(SRCS:src/%.c=...)`	Collect all sources automatically; map `src/foo.c` to `build/foo.o`. New files need no Makefile edit.
`$(BUILD)/%.o: src/%.c | $(BUILD)`	Pattern rule: how to build any object from its source. After `|` is an order-only prerequisite: the directory must exist but its timestamp does not trigger rebuilds.
`$@`, `$<`, `$^`	Automatic variables: the target, the first prerequisite, all prerequisites.
`-MMD -MP` and the last line	GCC writes a `.d` file listing the headers each source includes; `-include` loads them, so editing a header rebuilds the files that include it. Without this, stale objects cause baffling bugs.
`LIB_OBJS`	Every object except `main.o`: unit tests link against these, since `main.c` defines `main`.

> **PITFALL — the classic Makefile error**
> Make requires recipe lines to begin with a real TAB. Copying code from a PDF or an editor that converts tabs to spaces gives `Makefile:NN: *** missing separator. Stop.` Check with `cat -A Makefile`: recipe lines must start with `^I`.

shell: using the Makefile

```
make                                # debug build with sanitizers (default)
make clean && make SAN= OPT=-O2     # optimised build, no sanitizers: for release and valgrind
make test                           # unit tests, then integration tests
```

The Makefile does not notice a changed `SAN` or `OPT`, so always run `make clean` when switching between the two builds.

0.5 Git workflow

shell: the commands you need

```
git status                    # what changed?
git add -p                    # stage hunks deliberately (or: git add src/lexer.c)
git commit -m "Add tokenizer: words and operators"
git log --oneline             # your project history
git diff                      # unstaged changes
git restore src/lexer.c       # discard unstaged edits to a file
git switch -c try-ampersand   # branch for an experiment
git tag v1.0                  # mark the release (Stage 10)
```

- **Commit when something works**, with `make` warning-free and tests passing: at least once per task in each stage. Small commits are how you find a bug later (`git bisect` automates the search) and how a reviewer reads your progress.
- **Messages**: imperative, specific. "Fix leak in tokenize error path" beats "fixes".
- **Every bug gets a test first, then a commit** containing both test and fix. This habit is Stage 9 in miniature.
- **Do not commit** binaries, `build/`, editor files or core dumps.

0.6 First commit: a stub that builds

Create `src/main.c` containing the hello program, run `make`, and confirm: zero warnings, `./mysh` prints the line. Then break things on purpose, once each, and read the message: delete the `#include`; call a function that does not exist; replace a recipe TAB with spaces; write `int x; return x;`. Recognising these messages now saves hours later.

Completion criteria

	Observable result
[ ]	`git log --oneline` shows at least one commit; `git status` is clean.
[ ]	`make` prints no warnings; `./mysh` runs.
[ ]	`make clean` removes `build/` and `mysh`; a second `make` rebuilds everything.
[ ]	You can say, without looking, why `parser.h` is not on the gcc command line and what `-c` does.

CHECKPOINT — work these out before moving on

- You edit `util.h` and run `make`. Which objects are rebuilt, and which Makefile feature makes that true?
- Why does a unit test link `LIB_OBJS` but not `main.o`?
- A function body is accidentally placed in `lexer.h`, and both `lexer.c` and `parser.c` include it. Which phase fails, and with which message?

Hint: Trace what the preprocessor produces for each .c file, then ask how many object files end up containing a definition.

3. Debugging and testing toolkit

Read this once now and come back to it whenever something misbehaves. It answers "which tool, when, and how do I read what it says".

3.1 Which tool for which symptom

Symptom	Probable cause	First tool
Segmentation fault, garbage output	NULL or dangling pointer, missing NULL terminator on `argv`, off-by-one	AddressSanitizer report; then GDB `bt`
Sanitizer says "heap-buffer-overflow"	Write/read past an allocation (often `malloc(strlen(s))` without the `+1`)	Read the three stack traces (3.3)
LeakSanitizer lists leaks at exit	A `free` is missing, usually on an error path	The "allocated from" trace shows where; ask who should own it
Shell hangs after a pipeline	A pipe write end is still open somewhere, so the reader never sees EOF	`ls -l /proc/<pid>/fd` of the stuck process; `strace -f`
`Too many open files`	Descriptor leak (forgot `close`)	`ls /proc/<shell pid>/fd | wc -l` before and after a command
`defunct` processes in `ps`	Child never reaped with `waitpid`	`ps -o pid,stat,cmd --ppid <shell pid>`
Output twice, or in the wrong order	Unflushed stdio buffer duplicated by `fork`, or not flushed before `dup2`	Send output to a pipe (`| cat`); add `fflush`
A second prompt/copy of the shell appears after "command not found"	Child did not `_exit` after a failed `execvp`	Read the child code path
Ctrl-C kills the shell, or does nothing	No handler / `SA_RESTART` set / SIGINT ignored in the shell	Stage 8 behaviour table
Passes under ASan, fails in release (or the reverse)	Uninitialised variable or other undefined behaviour	Run both builds; UBSan; `-Wall -Wextra`
A test passes alone but fails in the suite	Shared state between tests (cwd, leftover files)	Give each test a fresh temp directory

3.2 A debugging routine that works

1. **Reproduce** with the smallest input that still fails, ideally a single line fed to `./mysh`.
2. **Capture it as a test** (a line in `run_integration.sh` or a `CHECK` in a unit test). A bug without a test comes back.
3. **Form a hypothesis** from the evidence (message, symptom, last change) before touching code.
4. **Pick the tool that tests the hypothesis**: sanitizer for memory, GDB for control flow and values, `strace` for system calls, `git diff` or `git bisect` for "it used to work".
5. **Fix, rerun the whole suite, commit test and fix together.**

3.3 Reading sanitizer reports

AddressSanitizer (ASan) replaces `malloc`/`free` and checks memory accesses; LeakSanitizer (LSan) runs at exit on Linux and lists allocations nobody freed; UndefinedBehaviorSanitizer (UBSan) checks for things like signed overflow. All are enabled by `-fsanitize=address,undefined`. They find memory-safety bugs at the moment they happen, with line numbers, rather than letting them corrupt something that crashes later. They do *not* find logic errors, descriptor leaks, zombies or deadlocks. Real, trimmed examples:

buggy.c (fragment)

```
char *dup(const char *s) {
    char *d = malloc(strlen(s));      /* BUG: no room for '\0' */
    strcpy(d, s);
    return d;
}
```

AddressSanitizer output (trimmed)

```
==4969==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000015 ...
WRITE of size 6 at 0x502000000015 thread T0
    #0 0x... in strcpy ../../../../src/libsanitizer/asan/asan_interceptors.cpp:563
    #1 0x... in dup /tmp/snip/bug1.c:5           <-- first frame in YOUR code: the bad write
    #2 0x... in main /tmp/snip/bug1.c:8
0x502000000015 is located 0 bytes after 5-byte region [0x502000000010,0x502000000015)
allocated by thread T0 here:
    #0 0x... in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x... in dup /tmp/snip/bug1.c:4           <-- where the too-small block came from
```

- **Line 1** names the bug class: `heap-buffer-overflow`, `heap-use-after-free`, `stack-buffer-overflow`, `double-free`.
- **First stack** is where the bad access happened. Skip library frames; read the first line of yours.
- **"located N bytes after a M-byte region"** says: you wrote just past the end of a block of size M. 0 bytes after 5, writing 6 bytes, with a string of length 5: the terminator did not fit.
- **"allocated by" / "freed by"** stacks show the block's history. For use-after-free, the "freed by" stack tells you who gave up ownership too early.

LeakSanitizer output (trimmed)

```
==4977==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 16 byte(s) in 1 object(s) allocated from:
    #0 0x... in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x... in main /tmp/snip/bug2.c:3
```

A *direct leak* is a block with no pointer to it left anywhere. The trace points at the allocation; the fix is to decide who owns that block and where it should be freed, usually on an error path you did not test. LSan only reports at exit and only unreachable memory, so a global that still points at a block is not a leak.

UndefinedBehaviorSanitizer output (a single line, file:line:column)

```
bug4.c:3:66: runtime error: signed integer overflow:
    2147483647 + 1 cannot be represented in type 'int'
```

> **NOTE — sanitizers and fork**
> A forked child that calls `exit()` runs the exit handlers and LeakSanitizer's end-of-process check too, which produces confusing duplicate reports (observed while preparing this guide). Children created for `execvp` should leave with `_exit()`; Stage 4 explains why.

3.4 GDB essentials

Command	Effect
`gdb ./mysh`	Start GDB on the executable (build with `-g`, which the Makefile does).
`run < script.txt`	Start the program; redirect a test script into it.
`break lexer.c:57`, `break tokenize`	Stop at a line or function. `info breakpoints`, `delete N` manage them.
`next` / `step` / `finish` / `continue`	Over a line / into a call / run until the current function returns / resume.
`print x`, `print *tl`, `print tl->items[2]`	Show values; works on structs and pointers. `print argv[0]@3` shows three elements.
`x/s ptr`, `x/16xb ptr`	Examine memory as a string / as 16 hex bytes.
`bt`, `up`, `down`, `info locals`	Call stack and local variables: the first thing to look at after a crash.
`watch var`	Stop whenever a variable changes: finds "who overwrote this".
`set follow-fork-mode child`	After `fork`, debug the child instead of the parent (default is the parent). `set detach-on-fork off` keeps both; `info inferiors` lists them.

shell: GDB together with ASan

```
ASAN_OPTIONS=abort_on_error=1:detect_leaks=0 gdb ./mysh
```

`abort_on_error=1` turns an ASan report into a `SIGABRT`, so GDB stops there and `bt` shows the full stack with all your locals. Leak checking is switched off here because LSan does not work while a debugger is attached.

3.5 strace and valgrind (optional, install separately)

```
shell: watch descriptors and processes
```

```
strace -f -o trace.txt ./mysh < script.txt
grep -E 'pipe2|dup2|close\(|clone|execve|wait4' trace.txt
```

`-f` follows children. Look for a `pipe2` whose write end is never `close`d in some process, or for a `wait4` missing for a `clone`. Expect extra lines from the sanitizer runtime in an ASan build; the filter above removes most of them. On a build made without sanitizers, `valgrind --leak-check=full --error-exitcode=1 ./mysh < script.txt` gives an independent leak check. Sanitizers and valgrind cannot be combined in one build.

3.6 Extra compiler checks worth one pass in Stage 9

`-Wshadow -Wformat=2 -Wstrict-prototypes -Wmissing-prototypes -Wvla -Wwrite-strings -Wcast-qual` add checks for shadowed variables, format-string misuse, missing `static` on helpers and more. The reference build is clean under all of them, so a warning in your code is worth reading. `gcc -fanalyzer` (GCC 10+) is a static analyser that follows paths through your code; it is noisy and reports some false positives, so treat each warning as a question to answer, not an order.

3.7 Test strategy at a glance

Layer	Tool	What it covers	From stage
Unit	`tests/test_*.c` + `check.h`, one program per module	Tokenizer and parser on valid, invalid and edge-case input; memory under ASan	2
Integration	`tests/run_integration.sh`: script on stdin, compare stdout / stderr / status	Execution, built-ins, redirection, pipelines, exit statuses, error messages	4
Resource checks	Inside the integration script: `/proc/self/fd`, `ulimit -n`, `ps`	Descriptor leaks, zombies, many-pipeline stress	6-9
Robustness	`tests/test_fuzz.c` (random lines), long-line tests	No crash or leak on garbage input	9
Interactive	Manual checklist (and optional pty script)	Prompt, Ctrl-C, Ctrl-D	8
Regression	Every bug adds a test line	Bugs stay fixed	always

This is the right balance for a personal C project: no test framework to learn, a 20-line assertion header and one shell script, and every run under sanitizers so memory bugs surface without writing dedicated memory tests.

Stage 1. Skeleton, memory helpers and line input

Core	**Estimate:** 2.5 hours	**Needs:** Step 0

Objective

A running read-eval loop that reads lines of any length, copes with end of input, behaves sensibly both at a terminal and in a pipe, and leaks nothing. It does not execute anything yet; for now it prints each line back in brackets so you can see exactly what was read.

Concepts

Strings, pointers and arrays (essential)

A C string is a sequence of bytes ending in a NUL byte (`'\0'`). `strlen` counts the bytes before it; the storage needed is `strlen + 1`. A `char *` is only an address: it knows nothing about how much memory follows. `sizeof` on an array gives its size in bytes, but on a pointer it gives the size of the pointer (8 on x86-64), which is the root of a classic bug:

fragment

```
char buf[128];
char *p = buf;
sizeof(buf);   /* 128 : the whole array              */
sizeof(p);     /* 8   : just the pointer, NOT 128    */
/* fgets(p, sizeof(p), stdin) would read at most 7 bytes: the bug */
```

Heap allocation and ownership (essential)

`malloc(n)` returns `n` uninitialised bytes, or NULL; `calloc` zeroes them; `realloc(p, n)` resizes, possibly *moving* the block; `free(p)` releases it. The compiler does not track lifetimes. You do, by deciding for every block *who owns it* and therefore who frees it, exactly once. Two realloc rules to internalise now:

fragment: the two shapes of realloc

```
p = realloc(p, new_size);        /* WRONG: if realloc fails, p is lost -> leak     */

char *tmp = realloc(p, new_size);/* RIGHT in general: keep the old pointer valid   */
if (tmp == NULL) { free(p); /* report error */ }
else             { p = tmp; }
```

If another pointer pointed into the old block, it now dangles after a successful `realloc` that moved the data. Remember this in Stage 2.

> **DESIGN DECISION — the x-wrappers**
> **Abort on out-of-memory.** Checking every allocation and unwinding half-built structures multiplies error paths. For a shell, running out of memory is effectively unrecoverable, so `xmalloc` and friends print a message and `exit(1)`. This keeps the code short and is a common choice in small programs. The alternative, propagating failure to callers, is what a library must do. Be ready to state this trade-off in an interview.

Reading a line: fgets (standard C) and its sharp edges

`fgets(buf, n, stream)` reads at most `n-1` bytes, stops after a newline (which it keeps), always NUL-terminates, and returns NULL at end of file or on error. Two consequences: a line is *complete* only if its last byte is `\n`; and a long line arrives in several calls, so you must loop and grow your buffer. POSIX also offers `getline(&buf, &cap, stream)`, which does the growing for you. Implement the loop yourself first, to practise buffer sizing; afterwards read `man 3 getline` and compare.

Interactive versus non-interactive (POSIX)

`isatty(STDIN_FILENO)` is true when stdin is a terminal. Print the prompt only then, and to stderr (POSIX specifies that an interactive shell writes its prompt to standard error), so that `echo ls | ./mysh` and every automated test produce clean stdout.

Tasks

#	Task	Time
1.1	Write `util.h` / `util.c`: `xmalloc`, `xcalloc`, `xrealloc`, `xstrdup`.	20 min
1.2	Write `shell.h` with `shell_state` and `input.h` with `read_status`.	10 min
1.3	Write `read_line` in `input.c` (pseudocode below).	60 min
1.4	Write the temporary echo loop in `main.c`.	15 min
1.5	Run the Stage 1 tests with sanitizers; fix; commit.	35 min
1.6	Read `man 3 getline`; write down two differences from your loop.	10 min

Interfaces and data structures

src/util.h (complete)

```
#ifndef UTIL_H
#define UTIL_H
#include <stddef.h>
void *xmalloc(size_t n);
void *xcalloc(size_t count, size_t size);
void *xrealloc(void *p, size_t n);
char *xstrdup(const char *s);
#endif
```

src/shell.h (complete)

```
#ifndef SHELL_H
#define SHELL_H
typedef struct shell_state {
    int last_status;      /* exit status of the most recent command            */
    int exit_requested;   /* set by the `exit` built-in; main loop checks it   */
} shell_state;
#endif
```

src/input.h (complete)

```
#ifndef INPUT_H
#define INPUT_H
#include <stdio.h>
typedef enum { READ_OK, READ_EOF, READ_INTERRUPTED } read_status;
read_status read_line(FILE *in, char **line_out);
#endif
```

void *xmalloc(size_t n) (xcalloc, xrealloc, xstrdup follow the same contract)	
Responsibility	Allocate like `malloc`, but never return NULL for a non-zero request.
Parameters / return	`n` bytes. Returns a pointer to uninitialised memory. `xrealloc(p, n)` may move the block; `xstrdup(s)` returns a heap copy of `s`.
Modifies	Nothing. `xrealloc` invalidates the old pointer when it moves the block.
Memory ownership	The caller owns the result and must `free` it. `xrealloc` takes back the old pointer and returns the new one: the caller must replace its copy and must not use the old one.
Errors	On failure prints `mysh: fatal: out of memory` to stderr and calls `exit(1)`. Never returns NULL.
Called by	Every module.

`read_status read_line(FILE *in, char **line_out)`	
Responsibility	Read one line of any length from `in`, without its trailing newline.
Parameters / return	`in`: stream to read (stdin). `line_out`: receives the line. Returns `READ_OK` (line available), `READ_EOF` (nothing more to read) or `READ_INTERRUPTED` (a signal interrupted the read; nothing returned, caller should show a new prompt).
Modifies	`*line_out` always: a new string on `READ_OK`, otherwise `NULL`. May clear the stream's error flag after an interruption.
Memory ownership	On `READ_OK` the caller owns `*line_out` and must `free` it. On other results nothing is allocated.
Errors	A final line without a trailing newline is returned as a normal line. A real read error is reported with `perror` and treated as `READ_EOF`. Out of memory aborts (x-wrappers).
Called by	`main` only.

Examples and pitfalls

`xmalloc` is the template for the other three wrappers: write them yourself.

src/util.c (fragment: one of four)

```
void *xmalloc(size_t n) {
    void *p = malloc(n);
    if (p == NULL && n != 0) {          /* malloc(0) may legitimately return NULL */
        fputs("mysh: fatal: out of memory\n", stderr);
        exit(1);
    }
    return p;
}
```

read_line (pseudocode; the EINTR branch only fires from Stage 8)

```
read_line(in, line_out):
    *line_out = NULL
    cap = 128;  len = 0;  buf = xmalloc(cap)
    loop:
        r = fgets(buf + len, cap - len, in)          # append after what we already have
        if r is NULL:
            if error flag set and errno == EINTR:
                clearerr(in); free(buf); return READ_INTERRUPTED
            if error flag set:                    perror; free(buf); return READ_EOF
            # end of file
            if len == 0: free(buf); return READ_EOF
            break                                    # last line had no newline: keep it
        len += strlen(buf + len)                     # how much did fgets just add?
        if buf[len-1] == '\n': remove it (len--, NUL-terminate); break
        if len + 1 >= cap:  cap = cap * 2;  buf = xrealloc(buf, cap)   # line continues
    *line_out = buf;  return READ_OK
```

PITFALL — common mistakes

- **Off-by-one on the terminator.** `xmalloc(strlen(s))` instead of `+ 1`. ASan reports a heap-buffer-overflow "0 bytes after" the block (Chapter 3).
- **`sizeof` on a pointer** instead of the buffer capacity (shown above).
- **Forgetting to `free(line)` each iteration.** The loop runs fine; LeakSanitizer reports "N objects" leaked at exit.
- **Embedded NUL bytes.** `strlen` stops at the first NUL, so such input is mis-measured and lines can merge. Document it as a limitation rather than chasing it; `getline` returns the true byte count, which is one reason it is the better tool.
- **Prompt on stdout.** Then every test that compares stdout sees `mysh$` noise.

src/main.c (complete for this stage; needs <stdio.h> <stdlib.h> <unistd.h> and the two headers)

```
int main(void) {
    shell_state sh = {0};
    int interactive = isatty(STDIN_FILENO);
    while (!sh.exit_requested) {
        if (interactive) fputs("mysh$ ", stderr);
        char *line;
        read_status rs = read_line(stdin, &line);
        if (rs == READ_EOF) break;
        if (rs == READ_INTERRUPTED) continue;   /* nothing returns this until Stage 8 */
        printf("[%s]\n", line);                 /* TEMPORARY: replaced in Stage 2 */
        free(line);
    }
    return sh.last_status;
}
```

Testing

Command	Expected
`printf 'a\nb' | ./mysh`	Prints `[a]` and `[b]`. The last line has no newline and must not be lost.
`printf '\n\n' | ./mysh`	Prints `[]` twice (empty lines are valid input).
`./mysh < /dev/null`	Exits immediately with status 0 (check with `echo $?`).
`head -c 100000 /dev/zero | tr '\0' x | ./mysh | wc -c`	100003 (`[` + 100000 x + `]` + newline). Proves the buffer grows correctly.
Interactive: type `hello`, press Enter, then Ctrl-D	Prompt shows only on a terminal; `[hello]` appears; Ctrl-D ends the shell.
Whole run under ASan/UBSan (default build)	No sanitizer output at exit.

Further learning

- `man 3 fgets`, `man 3 getline`, `man 3 realloc`: what exactly is guaranteed. Essential.
- Why `fgets` keeps the newline and what `getline` returns (the length). Useful.
- Search terms: "realloc pitfalls", "C string NUL terminator off-by-one", "isatty". Useful.
- Stdio buffering modes (`setvbuf`). Optional now, needed in Stage 6.

Completion criteria

	Observable result
[ ]	`make` is warning-free; all six tests above give the stated results.
[ ]	A 100,000-character line is read intact; `printf` without a trailing newline still yields the last line.
[ ]	No ASan/LSan/UBSan output on any run.
[ ]	You can explain who owns `*line_out` after each possible return value.

CHECKPOINT — work these out before moving on

- The buffer starts at 128 bytes and doubles. For a line of n bytes, roughly how many `realloc` calls happen, and how many bytes are copied in total in the worst case? Compare with growing by a fixed 128 bytes each time.
- What does `fgets` return and what is in the buffer for the input `abc` with no newline and then end of file? And for a line of exactly `cap - 1` characters followed by a newline?
- Why must `read_line` set `*line_out = NULL` when it returns something other than `READ_OK`?

Hint: For the first, write the series of sizes and sum them; the doubling case is geometric, the fixed case arithmetic. For the others, trace the loop by hand with a tiny capacity such as 4.

INTERVIEW LINK

Growing an array by doubling and reasoning about its total cost (amortised analysis), plus the `realloc` ownership idiom, are standard C interview material. You have just written the building block you will reuse for the token list, and the habit of tracing a loop by hand with a tiny capacity is the fastest way to find off-by-one bugs under time pressure.

Stage 2. The lexer: from characters to tokens

Core	**Estimate:** 4 hours (incl. unit tests)	**Needs:** Stage 1

Objective

Turn one line into a list of tokens: words (with quotes and escapes resolved) and the operators `|`, `<`, `>`, `>>`. This stage also introduces the unit-test setup you will use for the rest of the project. After it, `main` prints the tokens instead of the raw line.

Concepts

Scanning with an index and a small state machine (essential)

Walk the line with an index `i`. At each position you are in one of a few *contexts*, and the context plus the current character decide the action. That is a state machine, and it is the most reliable way to write a tokenizer because every rule is one row in a table you can check against the tests.

Context	Character	Action
Between tokens	space or tab	Skip it.
Between tokens	`| <`	Emit a PIPE / REDIR_IN token; advance 1.
Between tokens	`>`	If the next character is also `>`, emit APPEND and advance 2; else emit REDIR_OUT and advance 1 (longest match first).
Between tokens	anything else	Start a word: begin an empty buffer and enter "in word".
In word	space, tab, `|`, `<`, `>`, or end of line	The word ends: emit WORD (even if the buffer is empty, see pitfall) and return to "between".
In word	`'`	Copy characters literally up to the next `'`. End of line first: error "unterminated single quote".
In word	`"`	Copy up to the next `"`; inside, `\"` gives `"` and `\\` gives `\`. End of line first: error.
In word	`\`	Copy the next character literally. End of line right after: error "trailing backslash".
In word	anything else	Append it to the buffer.

A growable string and a growable array (essential)

You need two dynamic containers. A string builder collects the characters of the current word; a `token_list` collects tokens. Both use the same idiom: keep `len` and `cap`; when `len == cap`, double `cap` and `xrealloc`. Write the pattern once for the builder and once for the list; resist writing a generic container.

Ownership of token text and aliasing (essential)

Each WORD token owns its `text` (heap string, never NULL, possibly empty). Operator tokens have `text == NULL`. `free(NULL)` is defined to do nothing, so cleanup can free `text` unconditionally. A pointer into `items` such as `token *t = &tl->items[k];` is invalidated by the next `push` if it reallocates: this is *aliasing* of a moved object, the same dangling-pointer error as before, in a new disguise. Use indices, or take the pointer only after the last push.

Smaller C points

- `static` on a function gives it internal linkage: it is invisible to other files. Use it for helpers like `push` and `is_blank`.
- `isspace(c)` has undefined behaviour for negative `char` values. Write your own `is_blank` for space and tab; it is also clearer about what counts as a separator.
- Reading `line[i + 1]` is safe only if you already know `line[i] != '\0'`; otherwise you step past the terminator.

- `goto fail;` to a single cleanup label is a normal, accepted C idiom for releasing partial results on error paths. The alternative, repeating cleanup at each error, is how leaks happen.
- Return `const char *` error messages that point at string literals: static storage, no allocation, nothing to free.

Plan before you code

Do these three steps on paper first. They take 20 minutes and prevent most of the debugging.

1. **Trace by hand** `echo "a b">f` through the table above, writing the context, `i`, the buffer and the emitted tokens after each step. You should end with `WORD(echo) WORD(a b) REDIR_OUT WORD(f)`.
2. **List the edge cases** before writing a line of code (the test table below is your list).
3. **Write the unit tests that express them**, then implement until they pass.

Tasks

#	Task	Time
2.1	Write `tests/check.h` and a first test (`ls -l` gives two words). Run `make test-unit`; it should fail to link until the lexer exists.	25 min
2.2	Declare the types and functions in `lexer.h`; write `token_list_init`, `push` and `token_list_free`.	30 min
2.3	Implement `tokenize` for words and whitespace only. Make the first tests pass. Commit.	40 min
2.4	Add operators (including `>>`). Commit.	30 min
2.5	Add quotes and backslash escapes, plus all error paths. Commit.	60 min
2.6	Write the remaining tests from the table; run under ASan; fix leaks. Commit.	40 min
2.7	Replace the echo in `main` with a loop that prints each token's type and text.	15 min

Interfaces and data structures

src/lexer.h (complete)

```
#ifndef LEXER_H
#define LEXER_H
#include <stddef.h>

typedef enum { TOK_WORD, TOK_PIPE, TOK_REDIR_IN, TOK_REDIR_OUT, TOK_REDIR_APPEND } token_type;

typedef struct {
    token_type type;
    char *text;            /* TOK_WORD: owned heap string (may be ""); operators: NULL */
} token;

typedef struct {
    token *items;          /* heap array, NULL when empty */
    size_t len;
    size_t cap;
} token_list;

void token_list_init(token_list *tl);
int  tokenize(const char *line, token_list *out, const char **err);
void token_list_free(token_list *tl);
#endif
```

`int tokenize(const char *line, token_list *out, const char **err)`	
Responsibility	Split `line` into tokens, resolving quotes and escapes.
Parameters / return	`line`: NUL-terminated input, read only. `out`: receives the tokens. `err`: receives a message on failure. Returns `0` on success (possibly zero tokens) or `-1` on a syntax error.
Modifies	`*out` always (initialises it first). `*err` only on failure. Does not modify `line`.
Memory ownership	Allocates the array and each WORD text. The caller owns them and releases them with `token_list_free(out)`. The tokens do not point into `line`: the line may be freed immediately after the call.
Errors	Unterminated single or double quote, trailing backslash. `*err` points to a string literal (do not free). On failure the function frees any partial result and leaves `*out` empty, so `token_list_free` is still safe.
Called by	`main`; unit tests; `test_parser.c` helper.

`void token_list_free(token_list *tl)`	
Responsibility	Release everything the list owns.
Parameters / return	`tl`: a list previously set up by `token_list_init` or filled by `tokenize`. No return value.
Modifies	Frees each `text`, then `items`; resets `*tl` to the empty state (so calling it twice is safe).
Memory ownership	Does not free `tl` itself (it lives on the caller's stack).
Errors	None.
Called by	`main` after `parse`; `tokenize` on its own error path; tests.

Internal helpers (not in the header, all `static` in `lexer.c`): a string builder (`sb_init`, `sb_putc`, with `data`/`len`/`cap`), `push(token_list *, token_type, char *text)` which takes ownership of `text`, `is_blank(char)` and `is_op(char)`.

Examples and pitfalls

tokenize (pseudocode)

```
 tokenize(line, out, err):
     token_list_init(out);  i = 0
     while line[i] != '\0':
         c = line[i]
         if is_blank(c):  i++; continue
         if c is one of | < > :  emit operator (check ">>" first); advance; continue
         sb = new string builder
         while line[i] != '\0' and !is_blank(line[i]) and !is_op(line[i]):
             switch line[i]:
                 case '\'': copy until closing quote   (no closing quote -> error)
                 case '"' : copy until closing quote, handling \" and \\   (none -> error)
                 case '\\': copy next char             (nothing after it -> error)
                 default  : append line[i]; i++
         push(WORD, sb.data)                    # ownership of sb.data moves to the list
     return 0
   on any error:  free sb.data;  token_list_free(out);  *err = "...";  return -1
```

src/lexer.c (complete: the growth idiom; note sizeof *tl->items)

```
 static void push(token_list *tl, token_type t, char *text) {
     if (tl->len == tl->cap) {
         tl->cap = tl->cap ? tl->cap * 2 : 8;
         tl->items = xrealloc(tl->items, tl->cap * sizeof *tl->items);
     }
     tl->items[tl->len].type = t;
     tl->items[tl->len].text = text;
     tl->len++;
 }
```

PITFALL — common mistakes

- **Empty quoted word.** `echo "" x` has three words; the second is the empty string. If you decide "a word exists" by checking that the buffer is non-empty, you silently drop it. The decision to emit must come from having *entered* a word, not from its length.
- **Quoted operators.** `echo ">"` must give a WORD with text `>`. Decide operator versus word by the token type you emit, never by comparing strings later.
- **Longest match.** `>>>` is `>>` then `>`. Checking `>` before `>>` gives three tokens.
- **Leaks on error paths.** An unterminated quote after several tokens must free the tokens and the half-built word. ASan/LSan will show you if you forgot.
- **Dangling pointer after growth.** Holding `&tl->items[k]` across a `push`.
- **Using `strtok`.** It cannot handle quotes, it modifies its input and keeps hidden global state.

Testing

tests/check.h (complete: all the "framework" you need)

```
#ifndef CHECK_H
#define CHECK_H
#include <stdio.h>
#include <string.h>

static int g_failures = 0;
static int g_checks = 0;

#define CHECK(cond) do { \
    g_checks++; \
    if (!(cond)) { \
        g_failures++; \
        fprintf(stderr, "%s:%d: CHECK failed: %s\n", __FILE__, __LINE__, #cond); \
    } \
} while (0)

#define CHECK_STR(got, want) do { \
    g_checks++; \
    if ((got) == NULL || strcmp((got), (want)) != 0) { \
        g_failures++; \
        fprintf(stderr, "%s:%d: expected \"%s\", got \"%s\"\n", __FILE__, __LINE__, \
                (want), (got) ? (got) : "(null)"); \
    } \
} while (0)

#define TEST_REPORT() do { \
    printf("%d checks, %d failed\n", g_checks, g_failures); \
    return g_failures ? 1 : 0; \
} while (0)
#endif
```

tests/test_lexer.c (fragment: the shape of every unit test)

```
#include "check.h"
#include "lexer.h"

static void test_simple_words(void) {
    token_list tl; const char *err = NULL;
    CHECK(tokenize("  ls   -l  ", &tl, &err) == 0);
    CHECK(tl.len == 2);
    CHECK(tl.items[0].type == TOK_WORD);
    CHECK_STR(tl.items[0].text, "ls");
    CHECK_STR(tl.items[1].text, "-l");
    token_list_free(&tl);
}
int main(void) {
    test_simple_words();
    TEST_REPORT();
}
```

Input	Expected	Verifies
`ls -l`	WORD(ls) WORD(-l)	Whitespace skipping
(empty) and	No tokens, return 0	Empty input is not an error
`echo 'a b' "c d" x'y'"z"`	echo, `a b`, `c d`, `xyz`	Quotes group; adjacent pieces concatenate

Input	Expected	Verifies
`a|b>>c<d>e`	a PIPE b APPEND c IN d OUT e (9 tokens)	Operators without spaces
`>>>`	APPEND then OUT	Longest match
`echo ">" '|'`	Three WORDs; types are TOK_WORD	Quoted operators are text
`echo "" x`	Three WORDs; second text is `""`	Empty word is kept
`echo a\ b "q\"r"`	echo, `a b`, `q"r`	Backslash escapes
`echo 'abc`, `echo "abc`, `echo abc\`	Return -1, `err` set, `tl.len == 0`, `token_list_free` still safe	Error paths do not leak
500 words separated by spaces	500 tokens, last is `w499`	Array growth past several doublings

Run with `make test-unit`. Every test runs under ASan/UBSan, so a leak or overflow fails the run even if all `CHECK`s pass.

Further learning

- Finite state machines and lexical analysis; the "maximal munch" rule. Useful.
- `man 3 isspace` and why `unsigned char` casts matter. Useful.
- POSIX Shell Command Language, section 2.2 (Quoting) and 2.4 (Token Recognition): what a real shell does and what you deliberately left out. Optional.

Completion criteria

	Observable result
[ ]	`make test-unit` reports all lexer checks passing, with no sanitizer output.
[ ]	Each of the three syntax-error inputs returns `-1`, sets `err`, and leaves an empty list.
[ ]	`echo "" x` and `echo ">"` behave as in the table.
[ ]	You can trace `a"b c"d` by hand and say how many tokens result and why.

CHECKPOINT — work these out before moving on

- What is the time complexity of `tokenize` in the length n of the line, and its extra space? What would change if `push` grew the array by one element each time?
- Why does `echo hi>f` need no special handling in this design, but would break a design that splits on spaces first?
- Which pointers become invalid when `xrealloc` moves `tl->items`, and how could you structure the code so none are held across a `push`?

Hint: Count how many times each input character is examined. For the third, list every variable that holds an address into the array.

INTERVIEW LINK

A tokenizer is the classic "parse a string with state" problem: valid parentheses, decode string, simplified path, atoi, run-length encoding. The habits from this stage (trace one example by hand, enumerate edge cases such as empty input and unterminated constructs, then code) are exactly what to apply when a timed problem starts.

Stage 3. Command representation and the parser

Core	**Estimate:** 3.5 hours (incl. unit tests)	**Needs:** Stage 2

Objective

Turn the token list into a `pipeline`: an array of `command`s, each with a NULL-terminated `argv` and a list of redirections. Reject malformed lines with clear messages. After this stage `main` can tokenize and parse a line and print what it understood.

Concepts

Nested structures and who owns what (essential)

A `pipeline` owns an array of `command`s; each `command` owns its `argv` array, every string in it, and its `redirs` array with every path. Freeing is therefore a nested loop, and one missed level is a leak. Draw the picture before coding:

```
the data structure for one line (every string and array is separately heap-allocated)
 input:  cat < in.txt | sort > out.txt

 pipeline { cmds ---> [0] command { argv ---> ["cat", NULL]      argc=1
                                    redirs -> [ {REDIR_IN,  "in.txt"} ]   nredirs=1 }
                      [1] command { argv ---> ["sort", NULL]     argc=1
                                    redirs -> [ {REDIR_OUT, "out.txt"} ]  nredirs=1 }
            ncmds = 2 }
```

argv: the NULL terminator is part of the contract

`execvp(file, argv)` finds the end of the argument list by looking for a NULL pointer. Your `argv` therefore needs `argc + 1` slots, with the last one NULL. The same terminator lets built-ins safely look at `argv[1]` when there is no argument.

Count first, then allocate; zero-initialise for safe cleanup

The lexer grew its array dynamically because the size was unknown. Here the size is known cheaply: one pass over the tokens of a command counts words and redirections, then you allocate exact arrays and fill them. Allocate with `xcalloc` so unfilled slots are NULL/0: if a later command fails to parse, `pipeline_free` can still walk the half-built structure safely.

> **DESIGN DECISION — copy versus steal**
> **Deep copy, not ownership transfer.** `parse` copies each word with `xstrdup`. The token list and the pipeline are then independent: the tokens can be freed right after `parse` returns, and no string is ever freed twice. Transferring ownership (moving the pointers and setting `token.text = NULL`) avoids the copies but couples the lifetimes of the two structures and makes a double free one forgotten line away. For a shell, clarity beats saving a few allocations.

Syntax errors are about structure, not characters

Line	Result
(no tokens)	Success with `ncmds == 0`: do nothing. Not an error.
`ls | | ls` `ls | | wc`	Error: missing command (an empty segment between pipes or at an end).
`ls > ls <`	Error: expected a file name after redirection.
`ls > > f` `ls > | wc`	Error: the token after a redirection must be a WORD.
`> f`	Error: missing command (we require at least one word; Bash would create the file).
`> out echo hi`	Valid: redirections may appear anywhere in a command; `argv` is `echo hi`.
`cmd > a > b`	Valid: both redirections are recorded and applied in order (Stage 6).

Tasks

#	Task	Time
3.1	Write `command.h` and `pipeline_free` in `command.c`.	25 min
3.2	Write `test_parser.c` with the `parse_str` helper and the valid-case tests. Commit.	35 min
3.3	Implement `parse_command` (a static helper handling tokens `[start, end)`): count, validate, allocate, fill.	60 min
3.4	Implement `parse`: count pipes, allocate commands, split at PIPE tokens, call the helper, clean up on error.	40 min
3.5	Add the syntax-error tests; run under ASan; fix leaks. Commit.	30 min
3.6	Update `main`: tokenize, parse, free tokens, print a readable dump of the pipeline, free it (temporary).	20 min

Interfaces and data structures

src/command.h (complete)

```
#ifndef COMMAND_H
#define COMMAND_H
#include <stddef.h>

typedef enum { REDIR_IN, REDIR_OUT, REDIR_APPEND } redir_kind;

typedef struct {
    redir_kind kind;
    char *path;            /* owned heap string */
} redirection;

typedef struct {
    char **argv;           /* owned: argc strings followed by a NULL; argv[0] is the program */
    size_t argc;
    redirection *redirs;   /* owned array, NULL if nredirs == 0 */
    size_t nredirs;
} command;

typedef struct {
    command *cmds;         /* owned array of ncmds commands; NULL if ncmds == 0 */
    size_t ncmds;
} pipeline;

void pipeline_free(pipeline *p);
#endif
```

src/parser.h (complete)

```
#ifndef PARSER_H
#define PARSER_H
#include "command.h"
#include "lexer.h"
int parse(const token_list *tl, pipeline *out, const char **err);
#endif
```

`int parse(const token_list *tl, pipeline *out, const char **err)`	
Responsibility	Check the syntax of a token list and build the matching `pipeline`.
Parameters / return	`tl`: tokens from `tokenize` (read only). `out`: receives the pipeline. `err`: receives a message on failure. Returns `0` on success (`out->ncmds == 0` for an empty line) or `-1` on a syntax error.
Modifies	`*out` always; `*err` on failure. Never modifies `tl`.
Memory ownership	Allocates the entire pipeline as a deep copy; nothing in `*out` points into `tl`. The caller owns `*out` and releases it with `pipeline_free`. `tl` can be freed as soon as `parse` returns.
Errors	See the syntax table. `*err` is a static string literal. On failure everything built so far is freed and `*out` is empty.
Called by	`main`; `test_parser.c`.

`void pipeline_free(pipeline *p)`	
Responsibility	Free a pipeline and everything inside it.
Parameters / return	`p`: a pipeline from `parse`, possibly half-built or empty. No return value.
Modifies	Frees every `argv` string and array, every redirection path and array, and `cmds`; sets `cmds = NULL`, `ncmds = 0`. Safe to call twice.
Memory ownership	Does not free `p` itself.
Errors	None. Relies on unfilled slots being zero (hence `xcalloc`).
Called by	`main` after execution; `parse` on its error path; tests.

Examples and pitfalls

`parse_command (pseudocode; fill counters only after the allocation, so cleanup matches)`

```
 parse_command(tokens[0..n), command *c, err):
     nwords = 0; nredirs = 0
     for i in 0..n:                                  # pass 1: validate and count
         if tokens[i] is a redirection operator:
             if i+1 >= n or tokens[i+1] is not WORD:  *err = "expected a file name..."; return -1
             nredirs++;  i++                         # skip the file name too
         else: nwords++                              # it must be a WORD (PIPE was split off)
     if nwords == 0: *err = "missing command"; return -1
     c->argv   = xcalloc(nwords + 1, sizeof *c->argv)          # +1 for the NULL terminator
     c->redirs = nredirs ? xcalloc(nredirs, sizeof *c->redirs) : NULL
     for i in 0..n:                                  # pass 2: fill (deep copies)
         operator -> c->redirs[c->nredirs++] = {kind, xstrdup(next.text)}; i++
         word     -> c->argv[c->argc++] = xstrdup(tokens[i].text)
     return 0
```

PITFALL — common mistakes

- **`argv` without the NULL slot**: allocating `nwords` instead of `nwords + 1`. Works until `execvp` reads past the end; ASan catches the overflow in your fill loop only if you write the terminator.
- **Bounds check order**: test `i + 1 >= n` before reading `tokens[i + 1]`.
- **Cleanup on failure**: if command 2 of 3 fails, commands 0 and 1 are fully built. That is why `out->ncmds` is set before the loop and the array is zeroed: `pipeline_free` works on any partial state.
- **Double free**: freeing the tokens *and* trying to free strings you stole from them. With deep copy this cannot happen; keep it that way.
- **Forgetting a nesting level** in `pipeline_free` (the redirection paths are the usual casualty). LSan reports them as small leaks allocated in `xstrdup`.

Testing

Test	Expected
`ls -l /tmp`	1 command, `argc == 3`, `argv[3] == NULL`, no redirections
`ls -l | wc -l | cat`	3 commands with `argc` 2, 2, 1
`sort < in.txt >> out.txt`	1 command, `argc == 1`, 2 redirections: IN `in.txt`, APPEND `out.txt`
`> out echo hi`	`argv == {echo, hi}`, one REDIR_OUT
	Returns 0, `ncmds == 0`
`ls |`, `| ls`, `ls | | wc`, `ls >`, `ls > > f`, `ls > | wc`, `> f`, `ls <`	Each returns -1, `err != NULL`, and `pipeline_free` on the result is safe

The parser tests call `tokenize` first through a small helper `parse_str(line, &pipeline, &err)`; that is acceptable because the lexer is already tested. Keep every test under ASan: leaks in the error paths are the most likely bug of this stage.

Further learning

- Recursive-descent parsing and grammars (EBNF): your grammar from Chapter 1 maps almost directly to these two functions. Useful.
- `calloc` versus `malloc` + `memset`; why `sizeof *ptr` is safer than `sizeof(type)`. Useful.
- POSIX Shell Command Language, section 2.9 (Shell Commands) for the real grammar. Optional.

Completion criteria

	Observable result
[ ]	`make test-unit` passes lexer and parser tests under ASan/UBSan, no leaks.
[ ]	All eight malformed inputs return -1 with a message, and `main` prints it as `mysh: syntax error: ...` without exiting.
[ ]	`main` prints a correct dump for `cat < in.txt | sort > out.txt`.
[ ]	You can say which function frees `argv[i]` and when, and why the tokens can be freed before the pipeline is used.

CHECKPOINT — work these out before moving on

- If `parse` stole the token strings instead of copying, which line in `main` would have to change, and what could go wrong?
- What is the complexity of `parse` in the number of tokens? How many allocations does a command with w words and r redirections make?
- In `ls > | wc`, which check rejects the line, and why is it enough to look one token ahead?

Hint: Draw both lifetimes on a timeline of main's loop body. For counting allocations, list each xcalloc and xstrdup call.

INTERVIEW LINK

Freeing nested heap structures, deciding between copying and transferring ownership, and keeping partially built structures safe to free are the substance of many C questions (linked lists, trees, adjacency lists). Being able to say "here is who owns this and here is where it is released" is a strong interview signal.

Stage 4. Running external commands

Core	**Estimate:** 3.5 hours (incl. integration harness)	**Needs:** Stage 3

Objective

Execute a single command by creating a child process, replacing it with the requested program, waiting for it, and turning how it ended into an exit status. After this stage `mysh` is a working, if minimal, shell: `ls -l`, `sleep 1`, `echo hi` all work, errors are reported with the right status, and you have the integration test harness that all later stages extend.

Concepts

Processes, fork and exec (POSIX; essential)

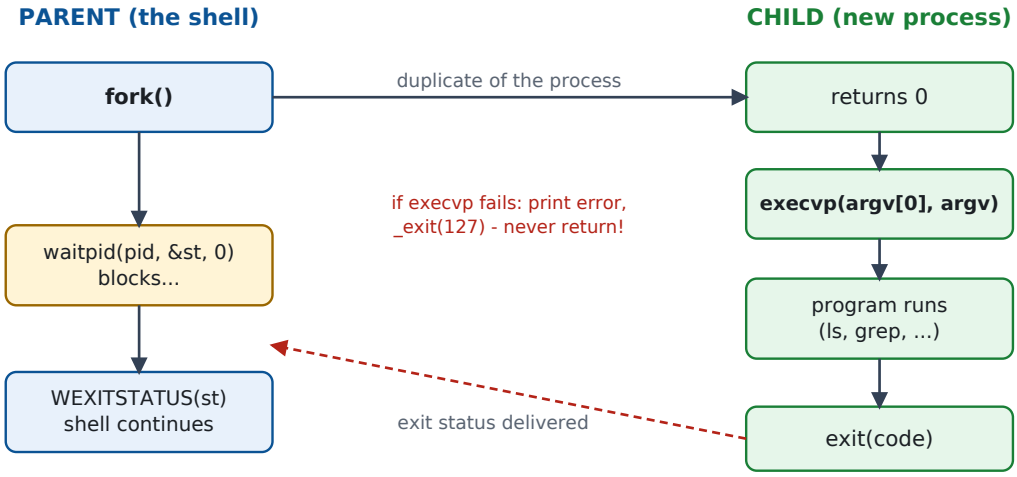


Figure 3. fork duplicates the shell; exec replaces the duplicate with the new program; waitpid collects the result.

- `fork()` creates a near-identical copy of the calling process. It returns *twice*: `0` in the child, the child's pid in the parent, and `-1` (parent only) if it failed. Memory is copied lazily by the kernel (copy-on-write), so it is cheap, but from the program's point of view the child has its own copy of every variable and open file descriptor table.
- `exec` replaces the calling process's program with a new one. On success it *never returns*; the code after it runs only if it failed. The process id stays the same; memory, but not open descriptors, is replaced.
- `waitpid(pid, &status, 0)` blocks until that child ends and returns its encoded status. Until a parent waits, a finished child remains a *zombie* (visible as `defunct` in `ps`) holding a process-table slot.

Function	Arguments	PATH search?	Notes
`execl`	path, then arg0, arg1, ..., NULL	no	Fixed, known-at-compile-time argument lists
`execv`	path, `argv[]`	no	Argument vector built at runtime
`execvp`	file, `argv[]`	yes	**Use this one**: the `p` searches `PATH` like a shell does
`execve`	path, `argv[]`, `envp[]`	no	The actual Linux system call; others are C-library wrappers
`execvpe`	file, `argv[]`, `envp[]`	yes	GNU extension, not POSIX: avoid

Decoding the wait status (POSIX; essential)

`status` from `waitpid` packs several facts in bits; never compare it directly. Use the macros: `WIFEXITED(st)` then `WEXITSTATUS(st)` for a normal exit, `WIFSIGNALED(st)` then `WTERMSIG(st)` if a signal killed it. Shell convention (not a C rule) reports the second case as `128 + signal number`: `139` for SIGSEGV, `130` for SIGINT.

Exit statuses for failure to run

Situation	`errno` from `execvp`	Message	Child exits with
Program not found in PATH	`ENOENT`	`mysh: foo: command not found`	127
Found but not executable / is a directory	`EACCES`, others	`mysh: /etc/passwd: Permission denied`	126
`fork` fails	(in the parent)	`mysh: fork: <reason>`	status 1, no child

Why `_exit`, and why flush before fork (essential)

If `execvp` fails, the child is still running your shell's code with a copy of its state. It must report the error and terminate with `_exit(status)`. Using `exit()` instead runs exit handlers and flushes the child's copy of the stdio buffers, and under the sanitizers repeats the leak check (duplicate reports). Forgetting to terminate at all is worse. Compare:

flawed.c (fragment)

```
pid_t pid = fork();
if (pid == 0) {
    execvp(argv[0], argv);
    perror("execvp");
    /* BUG: no _exit. The child falls out of the if and carries on as a second parent. */
}
waitpid(pid, NULL, 0);
printf("parent code running, pid %d\n", (int)getpid());
```

observed output of the flawed version (the shell would show two prompts)

```
execvp: No such file or directory
parent code running, pid 4932            <-- the child, which should be dead
parent code running, pid 4931            <-- the real parent
```

The second hazard is buffered output. If stdout is a pipe or file (not a terminal) it is fully buffered. Text printed but not flushed before `fork` exists in both processes' buffers and appears twice. Call `fflush(NULL)` before every `fork`.

demo_fork.c (complete and compilable: experiment with it outside the project first)

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    char *argv[] = { "ls", "-l", NULL };
    pid_t pid = fork();
    if (pid < 0) {                      /* parent only: fork failed */
        perror("fork");
        return 1;
    }
    if (pid == 0) {                     /* child only */
        execvp(argv[0], argv);
        perror("execvp");               /* reached only if exec failed */
        _exit(127);
    }
    int status;                         /* parent only */
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;
    }
    if (WIFEXITED(status))
        printf("child exited with %d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("child killed by signal %d\n", WTERMSIG(status));
    return 0;
}
```

Tasks

#	Task	Time
4.1	Compile and run `demo_fork.c` with the project flags. Change `ls -l` to a missing program and to `/etc/passwd`; note statuses. Run it under `strace -f` if installed.	25 min
4.2	Write `executor.h` and a first `execute_pipeline` that handles exactly one command: `fflush(NULL)`, fork, child runs `child_exec`, parent waits.	50 min
4.3	Write `static void child_exec(const command *cmd)` (never returns): `execvp`, report error, `_exit(126 or 127)`. Keep it separate; Stage 7 reuses it unchanged.	25 min
4.4	Write `static int wait_for(pid_t pid)` returning the decoded status, retrying when `waitpid` fails with `EINTR`.	25 min
4.5	Finish the real `main` loop (below). Commit.	15 min
4.6	Create `tests/run_integration.sh` and the first tests; run under sanitizers; fix.	50 min
4.7	Debug anything that fails; commit with the tests.	20 min

Interfaces and data structures

src/executor.h (complete; final signature, used unchanged through Stage 10)

```
#ifndef EXECUTOR_H
#define EXECUTOR_H
#include "command.h"
#include "shell.h"
int execute_pipeline(shell_state *sh, const pipeline *p);
#endif
```

int execute_pipeline(shell_state *sh, const pipeline *p)	
Responsibility	Run every command of `p` and return the pipeline's exit status.
Parameters / return	`sh`: shell state (built-ins may change it). `p`: a parsed pipeline with `ncmds >= 1`. Returns a status 0-255: that of the last command, or 1 if the shell itself failed (fork, pipe).
Modifies	`sh` only through built-ins (`exit_requested`). Never modifies `p`. Changes the process's working directory only via a built-in `cd` in the parent.
Memory ownership	Borrows `p` (does not free or keep it). Allocates and frees its own temporary pid array. All descriptors it opens are closed before it returns.
Errors	System-call failures are reported on stderr as `mysh: <context>: <reason>`; the function still returns normally.
Called by	`main`.

static void child_exec(const command *cmd) (executor.c, not in the header)	
Responsibility	Run in a freshly forked child: replace the process with the program.
Parameters / return	`cmd`: a command with a non-NULL `argv[0]`. Never returns.
Modifies	The child process only. The parent is unaffected.
Memory ownership	Frees nothing: after `execvp` succeeds the old memory vanishes; on failure the child `_exit`s.
Errors	Prints the message from the table above and `_exit(127)` for `ENOENT`, otherwise `_exit(126)`. Saves `errno` before calling `fprintf`.
Called by	`execute_pipeline` (Stage 4); `spawn_stage` (Stage 7).

src/main.c (complete glue: the final loop; includes as before plus lexer.h parser.h executor.h)

```
int main(void) {
    shell_state sh = {0};
    int interactive = isatty(STDIN_FILENO);
    /* setup_signals();   <- added in Stage 8 */
    while (!sh.exit_requested) {
        if (interactive) fputs("mysh$ ", stderr);
        char *line;
        read_status rs = read_line(stdin, &line);
        if (rs == READ_EOF) { if (interactive) fputc('\n', stderr); break; }
        if (rs == READ_INTERRUPTED) continue;

        token_list tl; const char *err;
```

```
        int rc = tokenize(line, &tl, &err);
        free(line);                                 /* tokens hold their own copies */
        if (rc < 0) {
            fprintf(stderr, "mysh: syntax error: %s\n", err);
            sh.last_status = 2;
            continue;
        }

        pipeline pl;
        rc = parse(&tl, &pl, &err);
        token_list_free(&tl);                       /* pipeline holds its own copies */
        if (rc < 0) {
            fprintf(stderr, "mysh: syntax error: %s\n", err);
            sh.last_status = 2;
            continue;
        }

        if (pl.ncmds > 0) sh.last_status = execute_pipeline(&sh, &pl);
        pipeline_free(&pl);
    }
    return sh.last_status;
}
```

Testing: the integration harness

The harness runs `mysh` as a black box: it feeds a script on stdin and compares stdout, stderr or the exit status. It runs in a fresh temporary directory so tests cannot disturb each other, and every invocation has a `timeout`, so a hang becomes a failed test instead of a frozen terminal.

tests/run_integration.sh (complete scaffolding; make it executable with chmod +x)

```
#!/usr/bin/env bash
# Integration tests: feed a script to mysh on stdin, compare stdout / exit status.
# Run from the project root:  ./tests/run_integration.sh   (or: make test-integration)
set -u
MYSH=${MYSH:-"$(pwd)/mysh"}
umask 022
WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT
cd "$WORK" || exit 1

pass=0; fail=0
ok()        { pass=$((pass + 1)); }
fail_msg()  { fail=$((fail + 1)); printf 'FAIL: %s\n  want: %q\n  got:  %q\n' "$1" "$2" "$3"; }
sanitizer_hit() { grep -q 'Sanitizer' "$WORK/.err"; }

# expect_out "<script>" "<expected stdout>"
expect_out() {
    local script=$1 want=$2 got
    got=$(printf '%s\n' "$script" | timeout 10 "$MYSH" 2>"$WORK/.err")
    if sanitizer_hit; then fail_msg "$script" "clean stderr" "$(cat "$WORK/.err")"
    elif [ "$got" == "$want" ]; then ok
    else fail_msg "$script" "$want" "$got"; fi
}

# expect_status "<script>" N      -- the shell's own exit status after running the script
expect_status() {
    local script=$1 want=$2 got
    printf '%s\n' "$script" | timeout 10 "$MYSH" >/dev/null 2>"$WORK/.err"; got=$?
    if sanitizer_hit; then fail_msg "$script" "clean stderr" "$(cat "$WORK/.err")"
    elif [ "$got" -eq "$want" ]; then ok
    else fail_msg "$script" "$want" "$got"; fi
}

# expect_err "<script>" "<substring of stderr>"
expect_err() {
    local script=$1 want=$2 got
    got=$(printf '%s\n' "$script" | timeout 10 "$MYSH" 2>&1 >/dev/null)
    if [[ "$got" == *"$want"* ]]; then ok; else fail_msg "$script" "*$want*" "$got"; fi
}

# ---- tests go here ----

printf '\n%d passed, %d failed\n' "$pass" "$fail"
```

```
[ "$fail" -eq 0 ]
```

Why `$(...)` strips trailing newlines, and what that means for your expectations: the comparison ignores final newlines, which is what you want for most tests. Why `2>&1 >/dev/null` in that order: it sends stderr to where stdout currently points (the capture) *before* stdout is discarded.

Script (stdin of mysh)	Check	Expected
`echo hello`	stdout	`hello`
`echo a b c`	stdout	`a b c`
`echo 'a b'`	stdout	`a b`
`/usr/bin/printf '[%s]\n' '' x`	stdout	`[]` and `[x]` on two lines (empty argument kept)
(empty line, blank line)	stdout	nothing
`true false`	status	0 1
`nosuchcommand_xyz`	status / stderr	127; `command not found`
`/etc/passwd`	status	126
`sh -c 'exit 3'`	status	3
`sh -c 'kill -SEGV $$'`	status	139 (killed by signal 11)
`sh -c 'kill -INT $$'`	status	130
`echo "unterminated`	status / stderr	2; `syntax error`
`false` newline `exit`	status	1 (exit with no argument uses the last status, Stage 5)

PITFALL — common mistakes

- **Zombies**: forgetting `waitpid`. Check with `ps -o pid,stat,cmd --ppid <shell pid>` while the shell sits at its prompt: no `Z` entries.
- **Not NULL-terminated `argv`**: `execvp` reads garbage; ASan or a strange "command not found" shows it.
- **Using `exit` in the child**, or no `_exit` at all (shown above).
- **Reading `status` directly** instead of the macros: a child that exits with 3 gives `status == 768`.
- **A test command that reads stdin**: in non-interactive mode a child such as `cat` with no file would swallow the rest of your script. Always give such commands a file or a pipe.
- **Calling `wait()` instead of `waitpid(pid, ...)`**: it may reap a different child; harmless now, wrong in Stage 7.

Further learning

- `man 2 fork`, `man 2 execve`, `man 3 exec`, `man 2 waitpid`: read the RETURN VALUE and ERRORS sections. Essential.
- `strace -f ./demo_fork` to see `clone`, `execve`, `wait4` as the kernel sees them. Useful.
- POSIX Shell Command Language, section 2.8.2 (Exit Status for Commands): the 126/127 rules. Useful.
- Copy-on-write; `vfork` and `posix_spawn` as alternatives to fork + exec. Optional.
- OSTEP, chapter 5, "Interlude: Process API" (free online). Useful.

Completion criteria

	Observable result
[ ]	`ls -l`, `echo a "b c"`, `sleep 1` and `sh -c 'exit 3'` behave correctly interactively and in the harness.
[ ]	All rows of the test table pass, with no sanitizer output, within the timeout.
[ ]	After running 20 commands, `ps` shows no zombie children of the shell.
[ ]	A mistyped command prints `command not found` and the prompt returns (one prompt, not two).

CHECKPOINT — work these out before moving on

- `main` frees `line` right after `tokenize`; the child is created later. What does the child hold, how does that relate to the parent's memory, and why is it safe for the child to never free anything?
- A program does `for (int i = 0; i < 3; i++) fork();` How many processes exist at the end, counting the original? Why?
- The test `sh -c 'kill -SEGV $$'` expects 139. Which two macros and which arithmetic produce it?

Hint: For the first, think about what exec does to the address space. For the second, draw the process tree after each iteration.

INTERVIEW LINK

"Explain fork/exec/wait" and "how many processes does this code create" are standard systems interview questions. You now have a working program to point to when answering, plus the experience of what goes wrong (zombies, duplicated buffers, missing `_exit`).

Stage 5. Built-in commands and shell state

Core	**Estimate:** 2 hours	**Needs:** Stage 4

Objective

Add `cd`, `pwd` and `exit`. They are implemented inside the shell, and `cd` and `exit` must run in the shell process itself. This stage also settles how the shell state flows between `main`, the executor and the built-ins.

Concepts

Why some commands cannot be programs (essential)

A child inherits a *copy* of the shell's attributes (working directory, environment, open files) and can change only its own. If `cd` were an external program, it would change the directory of a process that is about to exit, and the shell's directory would stay as it was. The same applies to `exit`. `pwd` could be external (and is, as `/bin/pwd`); it is a built-in here for speed and as a simple, testable example.

Function pointers and a dispatch table (essential)

Each built-in has the same signature, so a table maps names to functions. Looking up a command is a loop over a small array; the executor then calls the function through the pointer. This is the C way of expressing "an interface" without any language support.

`src/builtins.c (complete skeleton: you write the three builtin_* functions above it)`

```
typedef int (*builtin_fn)(shell_state *sh, char *const argv[]);

static const struct { const char *name; builtin_fn fn; } table[] = {
    { "cd", builtin_cd }, { "pwd", builtin_pwd }, { "exit", builtin_exit },
};

builtin_fn find_builtin(const char *name) {
    for (size_t i = 0; i < sizeof table / sizeof table[0]; i++)
        if (strcmp(name, table[i].name) == 0) return table[i].fn;
    return NULL;
}
```

`sizeof table / sizeof table[0]` is the number of elements, computed by the compiler. It works because `table` is an array in scope, not a pointer.

Parsing a number properly with strtol (essential)

`atoi` cannot tell "0" from "abc". `strtol(s, &end, 10)` can: set `errno = 0` first; if `end == s` there were no digits; if `*end != '\0'` there is trailing junk; if `errno == ERANGE` it overflowed. Exit codes are then reduced to 0-255 with `& 0xFF`.

Command	Behaviour
`exit`	Exit with the last command's status (this also makes statuses testable: `false` then `exit` gives 1).
`exit 3`	Exit with 3.
`exit 300`	Exit with 44 (300 mod 256), as every Unix shell does.
`exit abc`	`mysh: exit: abc: numeric argument required`; status 1; the shell does *not* exit.
`exit 1 2`	`mysh: exit: too many arguments`; status 1; does not exit.
`cd`	Change to `$HOME` (error if `HOME` is not set).
`cd dir`	Change to `dir`. On failure: `mysh: cd: dir: No such file or directory`, status 1.
`cd a b`	`mysh: cd: too many arguments`, status 1.
`pwd`	Print the working directory followed by a newline; status 0.

Exit through the main loop, not through exit()

`exit` sets `sh->exit_requested = 1` and returns the code; `main`'s loop ends and returns `sh.last_status`. Calling `exit()` from deep inside a built-in works but skips the orderly path through `main` and is harder to test and extend. The built-in's return value is stored by `main` as `last_status`, so the exit code flows through the same channel as every other status.

Environment, errno and getcwd

- `getenv("HOME")` returns a pointer into the environment: never free or modify it.
- `getcwd(buf, sizeof buf)` fills a buffer you provide; `PATH_MAX` (from `<limits.h>`) is the usual size. Pass the real array size, and check for NULL.
- `strerror(errno)` turns an error number into text. Read `errno` immediately after the failing call.

Tasks

#	Task	Time
5.1	Write `builtins.h` and the table + `find_builtin`.	15 min
5.2	Implement `builtin_pwd` and `builtin_cd` (including `HOME` and error messages).	30 min
5.3	Implement `builtin_exit` with `strtol`.	25 min
5.4	In `execute_pipeline`: if `ncmds == 1` and `find_builtin(argv[0])` is non-NULL, call it in the parent and return its status.	15 min
5.5	Add the integration tests below; run under sanitizers.	25 min
5.6	Commit; update your notes on which statuses each built-in returns.	10 min

Interfaces and data structures

`src/builtins.h (complete)`

```
#ifndef BUILTINS_H
#define BUILTINS_H
#include "shell.h"
typedef int (*builtin_fn)(shell_state *sh, char *const argv[]);
builtin_fn find_builtin(const char *name);
#endif
```

`builtin_fn find_builtin(const char *name)`	
Responsibility	Look up a built-in by command name.
Parameters / return	`name`: `argv[0]` of a command (read only). Returns the function, or `NULL` if `name` is not a built-in.
Modifies	Nothing.
Memory ownership	Returns a pointer to code, not memory: nothing to free.
Errors	None. `NULL` is an ordinary answer.
Called by	`execute_pipeline` (Stage 5), `spawn_stage` (Stage 7), `run_builtin_in_parent` (Stage 6).

`static int builtin_cd | builtin_pwd | builtin_exit(shell_state *sh, char *const argv[])`	
Responsibility	Implement one built-in. All three share the `builtin_fn` signature.
Parameters / return	`sh`: shell state. `argv`: NULL-terminated, `argv[0]` is the name. Returns the command's exit status (0 on success).
Modifies	`cd`: the process working directory. `exit`: `sh->exit_requested`. `pwd`: nothing (prints to stdout).
Memory ownership	None: `cd` and `exit` borrow `argv`; `pwd` uses a stack buffer.
Errors	Print `mysh: <name>: ...` to stderr and return 1. Never print to stdout except `pwd`'s result.
Called by	Through `find_builtin`, from the executor.

Pitfalls

PITFALL — common mistakes
- **Running `cd` in a forked child** (for example if you fork before checking for built-ins): everything appears to work, but the directory never changes in the shell.
- **`argv[1]` when there is no argument** is fine only because `argv` is NULL-terminated. Check `argv[1] != NULL` before using it.
- **Treating `exit abc` as `exit 0`** (`atoi` semantics) and quitting by accident.
- **No tilde expansion**: `cd ~` fails because `~` is not expanded. Document it as a limitation.
- **Wrong buffer size**: `getcwd(buf, sizeof(buf))` is right for an array; with a pointer it would pass 8.

Testing

Script	Expected
`cd /tmp` newline `pwd`	`/tmp` on stdout
`cd /tmp` / `cd ..` / `pwd`	`/`
`cd /nonexistent_dir_xyz` / `pwd`	stderr contains `No such file or directory`; `pwd` still prints the original directory
`exit 0` / `echo not-reached`	No output: the second line is never run
`exit 7`; `exit 300`; `exit abc`	Shell status 7; 44; 1 (and for `abc`, the next line still runs)
`false` / `exit`	Shell status 1

Further learning

- `man 2 chdir`, `man 3 getcwd`, `man 3 strtol`, `man 3 getenv`. Essential.
- Process attributes inherited across `fork` (`man 2 fork`, "inherited" section). Useful.
- How Bash distinguishes special built-ins, and why POSIX lists them (section 2.14). Optional.

Completion criteria

	Observable result
[ ]	`cd`, `pwd` and `exit` behave as in the table, in the harness and interactively.
[ ]	`cd /tmp` followed by `pwd` shows the change persisted in the shell.
[ ]	No built-in is ever run in a forked child (for a single command).
[ ]	You can explain why `cd` cannot be an external program in two sentences.

CHECKPOINT — work these out before moving on
- Why can't `cd` be an external program? Name what a child inherits and what it can change.
- `strtol("12abc", &end, 10)` returns 12. What do `end` and `errno` hold, and how does your code treat this input?
- The built-in lookup is a linear scan. At roughly how many built-ins would you replace it, and with what? What would you give up?

Hint: Think about the cost of the scan against the cost of a hash table, and about what really dominates the runtime of one command line.

INTERVIEW LINK
`strtol`-based parsing, with all its edge cases (empty string, sign, overflow, trailing characters), is the C answer to "implement atoi" and is a favourite for timed rounds. A table of function pointers is the idiomatic answer to "how would you do polymorphism in C".

Stage 6. Redirection and file descriptors

Core	**Estimate:** 3.5 hours	**Needs:** Stage 5

Objective

Support `< file`, `> file` and `>> file` on any command, including built-ins, with correct error handling and no leaked descriptors.

Concepts

File descriptors (POSIX; essential)

A file descriptor is a small integer that indexes a per-process table of open files. Three are set up for you: `0` standard input, `1` standard output, `2` standard error. Programs such as `echo` and `cat` simply read fd 0 and write fd 1; they do not know or care where those lead. *Redirection is therefore just rearranging the table before `exec`.*

- `open(path, flags, mode)` returns the *lowest unused* descriptor number, or -1.
- `dup2(old, new)` makes `new` refer to the same open file as `old`, closing `new` first if it was open. It returns `new` or -1. If `old == new` it does nothing.
- `close(fd)` removes the entry. The underlying file stays open while any other descriptor refers to it.
- Descriptors survive `fork` and `exec`. That is why the child can arrange its redirections between the two calls, and why leaked descriptors travel into every program you run.

Step in the child, for `echo hi > out.txt`	fd 0	fd 1	fd 2	fd 3
After `fork` (inherited)	terminal	terminal	terminal	-
`fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666)`	terminal	terminal	terminal	out.txt
`dup2(fd, STDOUT_FILENO)`	terminal	**out.txt**	terminal	out.txt
`close(fd)`	terminal	out.txt	terminal	(closed)
`execvp("echo", ...)`	terminal	out.txt	terminal	-

open flags and the mode argument

Redirection	Flags for `open`	Target fd
`< file`	`O_RDONLY`	0
`> file`	`O_WRONLY | O_CREAT | O_TRUNC`	1
`>> file`	`O_WRONLY | O_CREAT | O_APPEND`	1

With `O_CREAT` you must pass a third argument, the permission bits for a new file. Use `0666`: the kernel subtracts the process `umask` (commonly 022), giving the familiar `644`. Omitting it compiles without error and creates files with random permissions (`open` reads an undefined value).

Where redirection happens (essential)

For an external command, in the *child*, after `fork` and before `exec`: the shell itself is untouched. For a built-in run in the shell process (Stage 5 path) there is no child to change, so you must redirect the shell's own descriptors temporarily: save them with `dup`, apply, run the built-in, flush `stdout`, restore with `dup2`, close the saves.

pseudocode: built-in with redirection in the shell process

```
run_builtin_in_parent(sh, cmd, fn):
    fflush(stdout)                                  # nothing buffered may land in the file
    saved_in  = dup(0);  saved_out = dup(1)         # check both: dup can fail (EMFILE)
    if apply_redirections(cmd) < 0:   status = 1
    else:  status = fn(sh, cmd->argv);  fflush(stdout)   # push builtin's output into the file
    dup2(saved_in, 0);  close(saved_in)             # restore, even after a failed redirection
    dup2(saved_out, 1); close(saved_out)
    return status
```

`stdout` is a C `FILE` with its own buffer on top of descriptor 1. If you restore fd 1 before the buffer is flushed, the text goes to the terminal instead of the file, or is lost. This is the reason for `fflush(stdout)` before the restore.

apply_redirections (pseudocode)

```
for each redirection r in cmd->redirs, in order:     # left to right, like a real shell
    choose flags and target fd from the table above
    fd = open(r->path, flags, 0666)
    if fd < 0:  print "mysh: <path>: <strerror(errno)>";  return -1
    if fd != target:                        # open may already have returned the target number
        if dup2(fd, target) < 0:  perror; close(fd); return -1
        close(fd)                                    # the duplicate is no longer needed
return 0
```

Tasks

#	Task	Time
6.1	Experiment in a small C file: `open` + `dup2` + `execlp("echo", ...)` writing into a file. Check the file.	25 min
6.2	Write `static int apply_redirections(const command *cmd)`: open, dup2, close, error messages.	45 min
6.3	Call it in the child path between fork and `child_exec`; on failure `_exit(1)`.	15 min
6.4	Write `run_builtin_in_parent` with save/restore. Use it from the single-built-in path.	40 min
6.5	Add the tests below (including the descriptor-leak check). Run under sanitizers.	45 min
6.6	Review every path: which descriptors are open at each `return`? Commit.	20 min

Interfaces and data structures

`static int apply_redirections(const command *cmd) (executor.c)`	
Responsibility	Perform every redirection of `cmd` on the current process's descriptors 0 and 1.
Parameters / return	`cmd`: read only. Returns `0`, or `-1` after printing `mysh: <path>: <reason>`. Stops at the first failure.
Modifies	The calling process's file descriptor table (fd 0 and/or 1). Opens files, creates or truncates them.
Memory ownership	No heap memory. Every descriptor it opens is closed before returning, success or failure.
Errors	Missing file, no permission, is a directory, `dup2` failure. The caller decides what failure means: the child `_exit(1)`s, the parent path returns status 1.
Called by	`run_builtin_in_parent`; the child side of `spawn_stage` (Stage 7).

`static int run_builtin_in_parent(shell_state *sh, const command *cmd, builtin_fn fn)`	
Responsibility	Run a single built-in in the shell process with its redirections applied, leaving the shell's own descriptors unchanged.
Parameters / return	The built-in `fn` to call. Returns its status, or 1 if a redirection failed.
Modifies	Temporarily fd 0 and 1; permanently whatever the built-in changes (`cd`, `exit_requested`). Flushes `stdout`.
Memory ownership	Holds two saved descriptors; closes them before returning.
Errors	If `dup` fails, report it and return 1 without running the built-in (otherwise you cannot restore).
Called by	`execute_pipeline` when the pipeline is one built-in command.

Pitfalls

PITFALL — common mistakes

- **Missing mode argument** to `open` with `O_CREAT` (random permissions).
- **Not closing `fd` after `dup2`**: the program runs fine, but each command leaks a descriptor. Visible with `ls /proc/self/fd` (below) and fatal after a few hundred commands.
- **`if (fd != target)`** missing: if `open` returns the target number itself (possible when that descriptor is closed), `dup2` is a no-op and the following `close` would close your redirection.
- **No `fflush(stdout)`** before restoring: output lost or in the wrong place.
- **Not restoring** after a failed redirection: the shell's stdout stays pointed at a file.
- **Order of redirections**: `cmd > a > b` creates both files; the last one wins. Applying them in reverse changes which file receives output.
- **`cat < f > f`**: the shell truncates `f` before `cat` reads it, so the output is empty. Real shells behave identically; it is a known gotcha, not a bug.

Testing

Script	Expected
`echo hello > f.txt` / `cat f.txt`	`hello`
`echo one > f.txt` / `echo two >> f.txt` / `cat f.txt`	`one` then `two`
`echo one > f.txt` / `echo two > f.txt` / `cat f.txt`	`two` (truncated)
`echo data > in.txt` / `cat < in.txt`	`data`
`cat < a.txt > b.txt` after creating `a.txt`	`b.txt` has the same content
`echo hi>g.txt` / `cat g.txt`	`hi`: no spaces needed
`echo "a > b"`	`a > b`: a quoted `>` is text
`echo x > a.txt > b.txt` / `cat b.txt` / `cat a.txt`	`x` once: `a.txt` was created empty, `b.txt` has `x`
`pwd > p.txt` / `echo after` / `cat p.txt`	`after` then the directory: built-in output went to the file, nothing leaked to the terminal
`echo x > m.txt` / `stat -c %a m.txt`	`644` (with `umask 022` set at the top of the script)
`cat < missing.txt`	status 1; stderr names `missing.txt`
`pwd > /nonexistent_dir/x` / `exit`	status 1; the shell survives and `pwd` still works afterwards
`ls /proc/self/fd | wc -l`	`4`: fds 0, 1, 2 and the directory handle `ls` itself opened. Anything higher is a descriptor leaked from your shell. (Linux-specific. Compare with the same command in Bash if unsure.)

For redirection to work with a pipeline in the same command, ordering matters; see Stage 7. For now, test each redirection on its own.

Further learning

- `man 2 open`, `man 2 dup`, `man 2 close`; read the sentences about the lowest-numbered descriptor. Essential.
- `man 3 setvbuf`, and why stdout is line-buffered on a terminal but fully buffered into a file or pipe. Useful.
- POSIX Shell Command Language, section 2.7 (Redirection). Useful.
- "open file description" versus descriptor (why two descriptors from `dup` share an offset). Optional.
- `O_CLOEXEC` and why library code uses it. Optional.

Completion criteria

	Observable result
[ ]	Every row of the test table passes, no sanitizer output.
[ ]	`ls /proc/self/fd | wc -l` prints 4 before and after a series of redirected commands.
[ ]	After a failed redirection on a built-in, the shell's output still goes to the terminal.
[ ]	You can draw the descriptor table before and after `dup2` for a given command.

CHECKPOINT — work these out before moving on

- Why must redirection for an external command be applied in the child, between `fork` and `exec`, and not in the shell before `fork`? What changes for a built-in, and why does that force the save/restore?
- Suppose descriptor 0 was closed when your shell started. What does `open` return, and which line of `apply_redirections` protects you from a bug?
- What would you change in the lexer, the `redirection` struct and `apply_redirections` to support `2>file`?

Hint: Follow the integer values of the descriptors step by step. For the last question, list which of the three places needs a new field.

INTERVIEW LINK

"How does a shell implement `>`" is a common question, and the answer is the table above. Knowing that descriptors are integers into a per-process table, and what `dup2` does to it, also explains pipes, sockets and daemons later.

Stage 7. Pipelines

Core	**Estimate:** 4.5 hours	**Needs:** Stage 6

Objective

Run `a | b | c` for any number of stages: all stages start together, each stage's output feeds the next stage's input, and the shell returns the last stage's status without leaving zombies or stray descriptors. This is the hardest stage; budget for debugging.

Concepts

The pipe (POSIX; essential)

`pipe(fds)` creates a kernel buffer with two descriptors: `fds[0]` to read, `fds[1]` to write. Data written is read in order. The rules that matter:

- A reader gets end-of-file only when *every* descriptor referring to the write end is closed, in *every* process. One forgotten copy and the reader waits forever.
- A writer to a pipe whose read ends are all closed gets `SIGPIPE` (default: terminate) or `EPIPE`. That is how `yes | head -1` stops `yes`.
- The buffer is finite (typically 64 KiB on Linux, see `pipe(7)`); a writer blocks when it is full and a reader blocks when it is empty.
- Create the pipe *before* `fork`, so both processes inherit both ends.

All stages run at the same time

A pipeline is not "run a, then b". All commands are started first and then waited for. If you wait for stage 1 before starting stage 2, any stage that produces more than the pipe buffer deadlocks: it blocks writing and nobody reads. The test `seq 1 100000 | wc -l` exposes that bug.

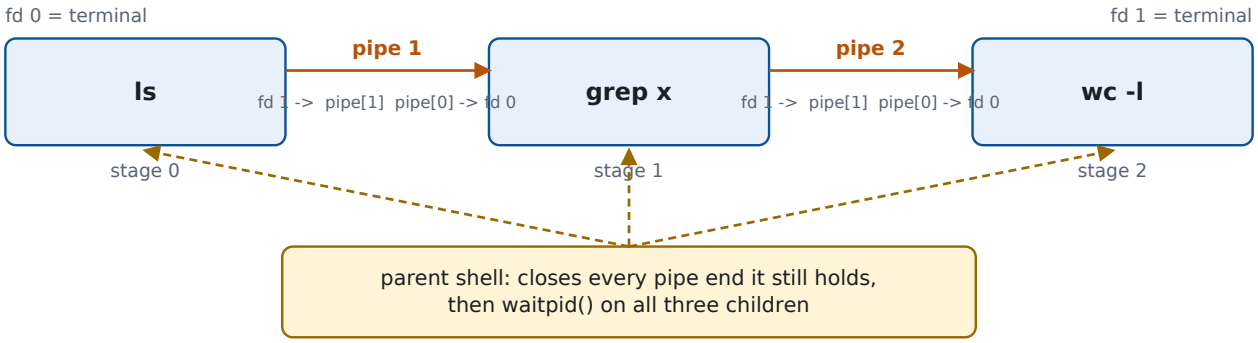


Figure 4. Descriptor wiring for ls | grep x | wc -l. Each pipe has one writer and one reader; the shell closes its own copies of every end.

Wiring one stage at a time

execute_pipeline (pseudocode; the algorithm, in full)

```
prev_read = -1                                  # read end of the previous pipe, or none
for i in 0 .. n-1:
    fds = {-1, -1}
    if i < n-1:  pipe(fds)                      # fds[0] = read end, fds[1] = write end
    pid = spawn_stage(sh, cmd[i], in_fd = prev_read, out_fd = fds[1], unused_fd = fds[0])
    if fork failed:  close any pipe ends we hold;  mark failure;  break
    pids[started++] = pid
    if prev_read != -1:  close(prev_read)       # the child has its own copy now
    if fds[1]  != -1:    close(fds[1])          # the parent never writes into the pipe
    prev_read = fds[0]                          # next stage reads from here (-1 after the last)
close prev_read if still open                   # only possible on the failure path
for i in 0 .. started-1:  status_i = wait_for(pids[i])      # wait for EVERY child
status = status of the last stage (or 1 if the shell failed to start some stage)
```

spawn_stage (pseudocode)

```
spawn_stage(sh, cmd, in_fd, out_fd, unused_fd):
    pid = fork();  if pid != 0: return pid              # parent (or -1)
    # ---- child ----
    reset SIGINT and SIGQUIT to default (Stage 8)
    if in_fd  != -1:  dup2(in_fd, 0);   close(in_fd)
    if out_fd != -1:  dup2(out_fd, 1);  close(out_fd)
    if unused_fd != -1:  close(unused_fd)    # our pipe's read end belongs to the next stage
    if apply_redirections(cmd) < 0:  _exit(1)   # AFTER wiring: `ls > f | wc` writes to f
    fn = find_builtin(cmd->argv[0])
    if fn:  status = fn(sh, cmd->argv);  fflush(stdout);  _exit(status)
    child_exec(cmd)                                     # never returns
```

Check the claim "no descriptor is left open" for each process by hand. In the child: `in_fd`, `out_fd` and `unused_fd` are closed (the first two after being duplicated). In the parent: after iteration `i`, it holds only the read end for stage `i + 1`. When the loop ends, it holds nothing.

Design rules that follow

- **Built-ins inside a pipeline run in the child** (`spawn_stage`), like Bash. `cd /tmp | cat` therefore does not change the shell's directory.
- **A single built-in still runs in the parent** (Stage 5/6 path): the one case where a built-in must affect the shell.
- **Pipeline status** is the status of the *last* command (no `pipefail`). The others are still waited for.
- **Redirection overrides the pipe**: `echo hi > f | cat` writes to `f` and `cat` sees an empty stream. This falls out of applying redirections after wiring.
- **`fflush(NULL)` before the first fork**, as in Stage 4.

Tasks

#	Task	Time
7.1	Write a 20-line experiment: `pipe`, fork a writer and a reader, `dup2`, `exec echo` and `cat`. Remove a `close` on purpose and watch it hang; fix it.	45 min
7.2	Extract the child-side code into `spawn_stage` (reusing `child_exec` and `apply_redirections`). Make single external commands use it with `-1, -1, -1`.	45 min
7.3	Implement the loop for N stages including the failure path. Wait for all children.	60 min
7.4	Make built-ins in pipelines run in the child; keep the single-built-in parent path.	20 min
7.5	Add the tests below. Run with and without sanitizers.	45 min
7.6	Debug hangs with the `/proc/<pid>/fd` and `strace -f` techniques. Commit.	55 min

Interfaces and data structures

`execute_pipeline` keeps the signature from Stage 4. Only its body changes.

`static pid_t spawn_stage(shell_state *sh, const command *cmd, int in_fd, int out_fd, int unused_fd)`	
Responsibility	Fork one pipeline stage and, in the child, wire its descriptors, apply redirections, and run it.
Parameters / return	`in_fd`: descriptor to become stdin, or -1 to leave it. `out_fd`: to become stdout, or -1. `unused_fd`: a descriptor the child must close (the read end of its own output pipe), or -1. Returns the child pid, or -1 if `fork` failed (parent only). In the child it never returns.
Modifies	Child: its descriptor table, signal dispositions, and then its whole program image. Parent: nothing.
Memory ownership	Borrows `cmd`; allocates nothing. In the parent the descriptors passed in remain the parent's to close.
Errors	Fork failure returned as -1 (the caller reports it and cleans up). In the child: failed redirection `_exit(1)`; failed exec `_exit(126/127)`.
Called by	`execute_pipeline`.

`static int wait_for(pid_t pid) (from Stage 4)`	
Responsibility	Wait for one specific child and decode its status.
Parameters / return	Returns `WEXITSTATUS`, or `128 + WTERMSIG`, or 1 if waiting failed.
Modifies	Reaps the child (it stops being a zombie).
Memory ownership	None.
Errors	Retries on `EINTR`; other failures reported with `perror`.
Called by	`execute_pipeline`, once per started child.

Pitfalls

PITFALL — common mistakes

- **Hang because a write end is still open.** Symptoms: `echo hi | cat` never finishes. Cause: the parent did not `close(fds[1])`, or a child did not close `unused_fd`. Diagnose: from another terminal, `ls -l /proc/<pid of the stuck cat>/fd` shows the pipe, and `strace -f` shows `read` blocked.
- **Closing the wrong end** in the parent (closing the read end you still need for the next stage).
- **Waiting inside the loop** before forking the next stage (deadlock with large output).
- **Not waiting for every child**: zombies for the earlier stages, and later stages' output may arrive after the prompt.
- **Fork failure half-way**: stages already started are waiting on a pipe whose writer never starts. Close what you hold, wait for the started children, return 1.
- **Reusing the previous `fds` array** after the last stage: initialise `fds` to `{-1, -1}` each iteration so "no pipe" is explicit.
- **Leaking the pid array** on the failure path (`free` before every return).

Testing

Script	Expected
`echo hello | tr a-z A-Z`	`HELLO`
`echo hi|cat`	`hi` (no spaces)
`echo hi | cat | cat | cat | cat`	`hi`
`/usr/bin/printf "b\na\nc\n" | sort | head -2`	`a` then `b`
`echo abc | tr a-z A-Z > up.txt` / `cat up.txt`	`ABC`: redirect on the last stage
`echo data > in.txt` / `cat < in.txt | tr a-z A-Z`	`DATA`: redirect on the first stage
`echo hi > f.txt | cat` / `cat f.txt`	`hi` printed once, by the second command
`yes | head -3`	`y` three times, and it terminates (SIGPIPE)
`seq 1 100000 | wc -l`	`100000`: more data than the pipe buffer, no deadlock
`pwd | cat` and `echo hi | cd /tmp` / `pwd`	Directory printed; the second `pwd` shows the *original* directory

Script	Expected
`true | false`, `false | true`, `echo x | nosuchcmd_xyz`	Statuses 1, 0, 127: last stage decides
`ls /proc/self/fd | wc -l` after a pipeline	4
300 lines of `echo x | cat | cat > /dev/null` under `ulimit -n 40`	No error output: no descriptor leak per pipeline
`true | true | true`, then a `ps` for zombies	No Z children of the shell

fragments for tests/run_integration.sh (verified: they fail if a close or a waitpid is removed)

```
# --- no fd leak across many pipelines: run 300 of them with a tiny fd limit
many=$(for i in $(seq 1 300); do echo 'echo x | cat | cat > /dev/null'; done)
got=$( (ulimit -n 40
        printf '%s\n' "$many" | timeout 30 "$MYSH" 2>&1 >/dev/null | head -c 200) )
if [ -z "$got" ]; then ok; else fail_msg "300 pipelines, ulimit -n 40" "no stderr" "$got"; fi

# --- zombie check: after several pipelines no defunct children may remain
zscript=$'true | true | true\ntrue | true\nsh -c \'ps -o stat= --ppid $PPID | grep -c Z\''
expect_out "$zscript" '0'
```

When a test hangs, the `timeout 10` in the harness turns it into a failure. Reproduce it by hand with `timeout 5 ./mysh` and a one-line script, then look at descriptors and system calls as described in Chapter 3.

Further learning

- `man 2 pipe`, `man 7 pipe` (capacity, atomic writes, EOF rules), `man 2 dup2`. Essential.
- `man 7 signal`: why SIGPIPE exists. Useful.
- POSIX Shell Command Language, section 2.9.2 (Pipelines). Useful.
- Process groups and sessions (`man 7 credentials`): needed for real job control, out of scope here. Optional.
- OSTEP, "Interlude: Process API", sections on pipes. Useful.

Completion criteria

	Observable result
[ ]	All pipeline tests pass under ASan/UBSan, within the timeout.
[ ]	The 300-pipeline test under `ulimit -n 40` passes; `ls /proc/self/fd | wc -l` still prints 4 afterwards.
[ ]	A pipeline with ten stages (`echo x | cat | cat | ...`) works.
[ ]	No zombies after any test. You can explain, for the pipeline `a | b | c`, which process holds which descriptors at the moment `c` is forked.

CHECKPOINT — work these out before moving on

- For N commands, how many calls to `fork`, `pipe` and `waitpid` happen, and what is the maximum number of pipe descriptors the shell itself holds at any one moment?
- At the moment stage 2 is forked in a 3-stage pipeline, list every process holding the write end of pipe 1. What would happen if one of them did not close it?
- In `yes | head -1`, trace which descriptor closes first, which process receives which signal, and why the pipeline terminates.

Hint: Make a table of processes and descriptors after each loop iteration. For the third, start from head exiting.

INTERVIEW LINK

Pipes and `fork` appear in many systems rounds ("implement `ls | wc`", "why does this program hang"). Reasoning about closure rules, and about concurrency without threads, transfers directly to socket servers and process pools.

Stage 8. Signals and robustness

Core (minimal)	**Estimate:** 2.5 hours	**Needs:** Stage 7

Objective

Make Ctrl-C behave like a shell: at the prompt it discards the line and shows a new prompt; while a command runs it terminates the command, not the shell. Ctrl-D ends the session; Ctrl-\ and Ctrl-Z must not damage the shell. Keep the implementation small.

Concepts

Signals and the terminal (POSIX; essential)

A signal is an asynchronous notification delivered to a process. Pressing Ctrl-C makes the terminal driver send `SIGINT` to the *foreground process group*. Your shell and all the children it starts without job control are in the same group, so a Ctrl-C reaches both the shell and the running command. The default action of `SIGINT` is to terminate; that is what would kill your shell.

Disposition	Meaning	Survives `exec`?
Default (`SIG_DFL`)	The signal's normal action (terminate, ignore, ...)	Yes (it is the default)
Ignored (`SIG_IGN`)	Discard the signal	**Yes**: programs you start inherit it
Handler (a function)	Run your function	No: reset to default

This is why the shell must *not* simply ignore SIGINT: every command would inherit "ignored" and Ctrl-C could not stop `sleep 30`. Use a handler in the shell (reset to default automatically on exec, and explicitly reset in the child before exec to cover the short window after `fork`).

sigaction, handlers and what a handler may do (essential)

- Use `sigaction`, not `signal`: its semantics are specified exactly; `signal` varies between systems.
- A handler can interrupt your program at any instruction, even in the middle of `malloc` or `printf`. It may call only *async-signal-safe* functions (listed in `signal-safety(7)`): `write` and `_exit` are; `printf`, `malloc`, `free` are not. Typical safe patterns: call `write`, or set a flag of type `volatile sig_atomic_t` and act on it later.
- Without `SA_RESTART`, a blocking system call that is interrupted by the handler returns -1 with `errno == EINTR`. You *want* this for the read at the prompt (so Ctrl-C gives a fresh prompt), and you must handle it where it is unwelcome: `waitpid`.

src/main.c (complete: signal setup; call setup_signals() once at startup)

```
#include <signal.h>
#include <string.h>
#include <unistd.h>

static void on_sigint(int sig) {
    (void)sig;
    ssize_t r = write(STDERR_FILENO, "\n", 1);   /* write(2) is async-signal-safe */
    (void)r;
}

void setup_signals(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = on_sigint;
    sa.sa_flags = 0;                  /* no SA_RESTART: blocking reads fail with EINTR */
    sigaction(SIGINT, &sa, NULL);
    sa.sa_handler = SIG_IGN;
    sigaction(SIGQUIT, &sa, NULL);
    sigaction(SIGTSTP, &sa, NULL);
}
```

Why it works: at the prompt, `read` returns `EINTR`, `fgets` returns NULL with the error flag set, and `read_line` turns that into `READ_INTERRUPTED` (the branch you sketched in Stage 1). While a command runs, the shell's `waitpid` is interrupted by the same Ctrl-C; the child dies from the default action, and your retry loop in `wait_for` calls `waitpid` again and collects the status (130).

Also write, in the child before exec (inside `spawn_stage` or just ahead of `child_exec`), a small function `reset_child_signals` that sets `SIGINT` and `SIGQUIT` back to `SIG_DFL` with `sigaction`. It is a handful of lines; model it on the setup above.

What the shell ignores, and the limitation that follows

- **SIGQUIT (Ctrl-\)** is ignored by the shell (otherwise it would dump core) and reset to default in children, so Ctrl-\ still kills a running program.
- **SIGTSTP (Ctrl-Z)** is ignored by the shell. Children inherit "ignored", so Ctrl-Z does nothing in programs started by `mysh`. Real job control (stopping and resuming jobs) needs process groups and `waitpid` with `WUNTRACED`, which is out of scope. Document this limitation.

Tasks

#	Task	Time
8.1	Read `man 7 signal-safety` and `man 2 sigaction` (flags, `SA_RESTART`).	20 min
8.2	Add `setup_signals` to `main.c` and call it first in `main`.	20 min
8.3	Write `reset_child_signals` and call it in the child path.	20 min
8.4	Make `read_line` return `READ_INTERRUPTED` on `EINTR` (clear the stream error!); make `wait_for` retry on `EINTR`. Check the `main` loop `continue`s without freeing anything twice.	35 min
8.5	Run the manual checklist below, then the automated status tests.	25 min
8.6	Optional: add the pty script. Commit.	30 min

Testing

Action in an interactive session	Expected
Press Ctrl-C at the empty prompt	`^C`, newline, a new prompt; shell alive
Type `echo partial` (no Enter), press Ctrl-C, type `echo fine` Enter	Only `fine` runs: the half-typed line was discarded
`sleep 30`, then Ctrl-C	Sleep dies immediately and a new prompt appears at once
`sleep 30 | cat`, then Ctrl-C	Both die; prompt returns; no zombies (`ps`)
Ctrl-Z at the prompt, then `echo ok`	Shell not stopped; `ok` printed
Ctrl-\ during `sleep 30`	Sleep dies (core dump message possible); shell survives
Ctrl-D at an empty prompt	Shell exits

Automated (these use signals sent by the child to itself, so they need no terminal and run in the normal harness):

Script	Expected shell status
`sh -c 'kill -INT $$'`	130
`sh -c 'kill -QUIT $$'`	131
`sh -c 'kill -SEGV $$'`	139

Optional automated check of the interactive behaviour, using a pseudo-terminal (Python 3 standard library only). Run from the project root after `make`: `python3 tests/pty_test.py`.

tests/pty_test.py (complete; verified against the reference build)

```
#!/usr/bin/env python3
"""Optional: drive ./mysh through a pseudo-terminal and press Ctrl-C / Ctrl-D."""
import os, pty, select, time

def expect(fd, needle, timeout=5):
    buf, end = b"", time.time() + timeout
```

```
    while time.time() < end and needle not in buf:
        if select.select([fd], [], [], 0.2)[0]:
            try:
                buf += os.read(fd, 4096)
            except OSError:
                break
    return needle in buf

pid, fd = pty.fork()
if pid == 0:
    os.execv("./mysh", ["./mysh"])

results = {}
results["prompt appears"] = expect(fd, b"mysh$ ")
os.write(fd, b"\x03")                          # Ctrl-C at an empty prompt
results["shell survives ^C at prompt"] = expect(fd, b"mysh$ ")
os.write(fd, b"sleep 30\n")
time.sleep(0.5)
os.write(fd, b"\x03")                          # Ctrl-C while a child runs
results["child killed, prompt returns"] = expect(fd, b"mysh$ ", timeout=3)
os.write(fd, b"echo still-alive\n")
results["shell still works"] = expect(fd, b"\r\nstill-alive\r\n")
os.write(fd, b"\x04")                          # Ctrl-D ends the shell
time.sleep(0.5)
done, _ = os.waitpid(pid, os.WNOHANG)
results["exits on ^D"] = (done == pid)

for name, ok in results.items():
    print("PASS" if ok else "FAIL", name)
raise SystemExit(0 if all(results.values()) else 1)
```

PITFALL — common mistakes

- **`printf` or `malloc` in the handler**: undefined behaviour, and an occasional deadlock that appears once a month.
- **`SA_RESTART` set** (or `signal()` on glibc, which restarts by default): Ctrl-C at the prompt appears to do nothing, because `read` simply resumes.
- **SIGINT set to `SIG_IGN` in the shell**: Ctrl-C then cannot kill any child either.
- **No `EINTR` retry around `waitpid`**: the shell stops waiting for a still-running child and prints the next prompt too early; the child later becomes a zombie.
- **Forgetting `clearerr(stdin)`** after an interrupted `fgets`: the error flag stays set and every later read fails.
- **Freeing `line` twice** or leaking it on the `READ_INTERRUPTED` path.

Further learning

- `man 2 sigaction`, `man 7 signal`, `man 7 signal-safety`. Essential.
- The self-pipe trick and `pselect`, the race-free way to wait for input or a signal. Optional.
- Process groups, sessions, controlling terminals and `tcsetpgrp`: what job control needs. Optional.
- POSIX Shell Command Language, section 2.11 (Signals and Error Handling). Useful.

Completion criteria

	Observable result
[ ]	The manual checklist behaves as in the table; the three status tests pass.
[ ]	Ctrl-C during a 3-stage pipeline leaves no processes behind and no zombies.
[ ]	The handler contains only `write`; there is no other signal-unsafe call in it.
[ ]	You can explain why SIGINT is handled, SIGQUIT is ignored in the shell but reset in children, and what Ctrl-Z does.

CHECKPOINT — work these out before moving on

- Why does Ctrl-C reach both the shell and its child? What would change if the child were placed in a new process group?
- Which of `write`, `printf`, `malloc`, `_exit`, `strlen` can you call from a handler? How do you find out for sure?
- If the handler called `free()` on a global buffer, describe a sequence of events that corrupts the heap.

Hint: For the first, read about foreground process groups. For the third, imagine the signal arriving while the main program is inside `free` or `malloc`.

INTERVIEW LINK
Retrying a system call on `EINTR`, and writing `volatile sig_atomic_t` flags, are details interviewers use to separate people who have written real C from people who have read about it.

Stage 9. Hardening and test consolidation

Core	**Estimate:** 2.5 hours	**Needs:** Stage 8

Objective

Turn "works on my examples" into "reliable". No new features: audit ownership and descriptors, stress the shell, fuzz the parser, make sure every bug you found has a test, and run everything in both build modes.

Tasks

#	Task	Time
9.1	Ownership audit: fill in the table below for every allocation in your code.	30 min
9.2	Error-path audit: for every system call in `executor.c` and `builtins.c`, check what happens when it fails.	30 min
9.3	Add `tests/test_fuzz.c` (below); run it. Fix anything it finds.	25 min
9.4	Stress and resource tests: very long line, many tokens, long pipeline, descriptor and zombie checks.	25 min
9.5	Run everything in both builds (sanitizers; `make clean && make SAN= OPT=-O2`). Optionally valgrind on the release build and the extra warnings from Chapter 3.	20 min
9.6	Review the regression tests list: every bug fixed so far has a line in the suite.	10 min

9.1 Ownership audit

Complete this table from your own code. A blank "freed by" or an "error path?" you cannot answer is where the next leak is.

Allocation	Allocated in	Owner afterwards	Freed by	Freed on error paths?
The input line	`read_line`	caller (`main`)	`main`, right after `tokenize`	tokenize error: yes, `main` frees before checking `rc`
Token texts and `items`	`tokenize`	caller	`token_list_free`	you fill in
String builder buffer	`tokenize` (static helper)	you fill in	you fill in	you fill in
`argv` array and its strings	`parse_command`	the `pipeline`	you fill in	you fill in
Redirection paths and array	you fill in	you fill in	`pipeline_free`	you fill in
`cmds` array	`parse`	caller	`pipeline_free`	parse error: yes (zeroed)
`pids` array	`execute_pipeline`	the function itself	you fill in	fork failure: you fill in

9.2 Error-path audit

Call	If it fails...	Required behaviour
`pipe`	Too many open files	Report; close prev read end; wait for started stages; status 1
`fork`	Out of processes	Same as above; close both ends of the new pipe
`open` (redirection)	Missing file, permission	Message with path and reason; child `_exit(1)` / built-in returns 1
`dup`, `dup2`	Descriptor limit	Do not run the built-in without a way to restore; close what you opened
`waitpid`	`EINTR` / `ECHILD`	Retry on `EINTR`; report others once
`chdir`, `getcwd`	Missing directory, permission	`mysh: cd: <dir>: <reason>`, status 1; shell continues
`execvp`	Not found, not executable	127 / 126 with the message; `_exit`, never return

Call	If it fails...	Required behaviour
`xmalloc` family	Out of memory	Message and `exit(1)`: the documented policy

9.3 Parser fuzz test

Random strings made from the interesting characters will hit combinations you did not think of: quotes unbalanced in odd places, operators at the ends, empty words. The test never executes anything, so it is safe. Under ASan/UBSan, any overflow or leak aborts the run.

`tests/test_fuzz.c (complete; 200,000 inputs in under a second; picked up by `make test-unit`)`

```
/* tests/test_fuzz.c: random garbage must never crash or leak in tokenize/parse */
#include "check.h"
#include "parser.h"
#include <stdlib.h>

int main(void) {
    static const char alphabet[] = "ab |<>'\"\\\t";
    srand(12345);                              /* fixed seed: failures reproduce */
    for (int iter = 0; iter < 200000; iter++) {
        char line[32];
        size_t len = (size_t)(rand() % 24);
        for (size_t i = 0; i < len; i++)
            line[i] = alphabet[rand() % (int)(sizeof alphabet - 1)];
        line[len] = '\0';

        token_list tl; pipeline p; const char *err;
        if (tokenize(line, &tl, &err) == 0) {
            if (parse(&tl, &p, &err) == 0) pipeline_free(&p);
            token_list_free(&tl);
        }
        g_checks++;
    }
    TEST_REPORT();
}
```

Do not fuzz the whole shell by feeding random bytes into `mysh`: random input can contain a valid command line. (If you do experiment, run in an empty temporary directory with `PATH` pointing at an empty directory.)

9.4 Stress and resource checks

Check	How	Expected
Very long line	`printf 'echo %s | wc -c\n' $(head -c 5000 /dev/zero | tr '\0' x) | ./mysh`	5001
Many arguments	`printf 'echo %s | wc -w\n' "$(seq 1 5000 | tr '\n' ' ')" | ./mysh`	5000
Long pipeline	`{ printf 'echo x'; for i in $(seq 50); do printf ' | cat'; done; echo; } | ./mysh`	`x`; no hang; no leaked descriptors
Descriptor leak	The `ulimit -n 40` test of Stage 7	Passes
Zombies	Several pipelines, then `ps -o stat= --ppid $PPID` from inside the shell	No `Z`
Sanitizers vs release	Run `make test` in both build modes	Same results

Further learning

- Fuzzing and property-based testing ("what must always hold for any input?"). Useful.
- `valgrind --leak-check=full --track-fds=yes`: also reports leaked descriptors. Useful.
- `man 7 proc` and `man 5 proc`: `/proc/<pid>/fd`, `/proc/<pid>/status`. Useful.
- Why ASan and valgrind are complementary: instrumentation at compile time versus emulation at run time. Optional.

Completion criteria

	Observable result
[ ]	`make test` passes in the sanitizer build and in the `SAN= OPT=-O2` build.
[ ]	The ownership table has no blanks and you can justify every row.
[ ]	Fuzz test, 300-pipeline test and long-line tests pass; extra warning flags from Chapter 3 produce no warnings in your code (or each is understood).
[ ]	Every bug you fixed during the project has a regression test in the suite.

CHECKPOINT — work these out before moving on

- Why does LeakSanitizer not report memory that is still reachable from a global at exit, and why is that sometimes misleading?
- Name three classes of bug that ASan and UBSan cannot find in this project, and the test that would find each.
- You see a leak report whose "allocated from" trace ends in `xstrdup` called from `parse_command`. Which two functions do you inspect first, and why?

Hint: For the second, think about descriptors, process lifecycle, and logic.

Stage 10. Documentation, repository and CV packaging

Core	**Estimate:** 2 hours	**Needs:** Stage 9

Objective

Make a stranger (a recruiter, an interviewer) able to understand, build, test and trust the project in five minutes, and prepare yourself to discuss it for thirty.

Tasks

#	Task	Time
10.1	Write `README.md` from the outline below. Include real outputs, not invented ones.	55 min
10.2	Add short comments where behaviour is not obvious (ownership rules in headers, why `_exit`, why `fflush` before `fork`, why no `SA_RESTART`).	20 min
10.3	Clean-clone check: `git clone . /tmp/mysh-check && cd /tmp/mysh-check && make && make test`. Fix anything that fails.	15 min
10.4	Tag `v1.0`; read the interview questions below aloud once, without notes.	30 min

README outline

README.md (outline; the fenced sections are yours to fill with real content)

```
# mysh: a small Unix shell in C

One paragraph: what it is, that it is a learning project, platform (Linux, GCC, Make).

## Features                      (only what the tests cover)
- Interactive and non-interactive use; quoting ('..', "..", \x); < > >> redirection
- Pipelines of any length; built-ins cd, pwd, exit; exit statuses (126/127/128+n)
- Ctrl-C handling

## Build and run
    make                # debug build with ASan/UBSan
    ./mysh
    make clean && make SAN= OPT=-O2    # optimised build

## Example session           (paste a real one)

## Tests
    make test            # N unit checks + M integration checks (give the real numbers)
What is covered; that everything runs under ASan/UBSan; how to run the optional pty test.

## Architecture              (short: module table and the data flow from Chapter 1)
Memory ownership in three sentences. Process model in three sentences.

## Design decisions          (3-5 bullets, each with the alternative you rejected)
abort on out-of-memory; deep copy in the parser; built-in in the parent only when alone;
Ctrl-C via handler not SIG_IGN; fgets loop instead of readline.

## Known limitations         (honest, specific)
No job control or `&`, no globbing, no variables or `$?`, no history or line editing,
no `~` expansion, no `2>`; Ctrl-Z ignored; NUL bytes in input unsupported; Linux only.
```

What makes a small systems project credible

Reviewers skim for evidence, not size. Roughly in order of weight:

1. **It builds and runs from a clean clone** with the documented commands. Nothing else matters if this fails.
2. **Tests exist, are automated, and run under sanitizers.** Unit tests for the lexer and parser, end-to-end tests for execution, and a stress or fuzz test show engineering habits.

3. **Resource discipline**: no leaks, no descriptor leaks, no zombies, and a short statement of how you checked.
4. **Clear structure**: small modules, readable names, ownership documented where it is not obvious.
5. **Honest scope**: a limitations section that names what is missing shows judgement. Overclaiming is the fastest way to lose an interviewer.
6. **A history of small, sensible commits** with tests next to fixes.

> **PITFALL — stay honest**
> Do not write "POSIX-compliant", "Bash-compatible" or "supports job control". Do not list features you have not implemented and tested, and do not inflate line counts or test counts: use `wc -l src/*.c` and your actual test output. Everything on your CV about this project should survive the question "show me where."

CV material

Replace every placeholder with your real number. Pick two or three bullets, not all:

- Implemented a Unix shell in C (about [N] lines) on Linux: tokenizer with quoting, parser into command structures, `fork`/`execvp`/`waitpid` execution, `<`, `>`, `>>` redirection and multi-stage pipelines via `pipe` and `dup2`.
- Designed explicit memory-ownership rules and verified them with AddressSanitizer, UndefinedBehaviorSanitizer and LeakSanitizer; [N] unit checks and [M] integration tests plus a parser fuzz test, all green in debug and optimised builds.
- Handled signals correctly (`sigaction`, `EINTR` retry, async-signal-safe handler), so Ctrl-C stops foreground jobs without killing the shell; diagnosed descriptor and zombie leaks with `/proc`, `strace` and `ulimit`.

Interview readiness: questions to answer aloud

You are done with this section when you can answer each in 30-60 seconds without looking at the code.

- Walk me through what happens from pressing Enter to the output appearing, including which process does what.
- Why must `cd` be a built-in? Which other built-ins must be, and which are only conveniences?
- Who owns the memory the parser returns? What happens if the lexer fails halfway?
- What does `fork` return, and what is shared or copied afterwards? Why `_exit` in the child?
- Draw the descriptors for `a | b | c`. What if the parent forgets to close the write end?
- Why do pipeline stages have to run concurrently? What deadlocks if you wait between forks?
- How is `>` implemented? Why does a built-in with `>` need save and restore?
- What does `waitpid` return in `status`, and how do you get 139 for a segfault?
- What is a zombie, and how did you check that your shell leaves none?
- Why a handler for SIGINT instead of ignoring it? Why is `printf` forbidden in a handler?
- What is `EINTR`, and where in your code does it matter?
- Why abort on out-of-memory? When is that the wrong design?
- Which bugs did the sanitizers find? Which did they not find, and what did?
- What would you add next (E1-E4), and what would it cost in the lexer, parser and executor?
- What are the shell's known limitations and why did you accept them?

A two-minute demo

demo script

```
./mysh
mysh$ echo "hello   world" | tr a-z A-Z
mysh$ ls | sort -r | head -3 > top.txt
mysh$ cat < top.txt
mysh$ cd /tmp
mysh$ nosuchcommand          # 127, clear message
mysh$ sleep 30               # press Ctrl-C: shell survives
mysh$ exit
make test                    # show the green summary
```

Completion criteria

	Observable result
[ ]	A fresh clone builds with `make` and passes `make test` using only the README instructions.
[ ]	The README states real test counts, real limitations, and no feature you did not test.
[ ]	Tag `v1.0` exists; the working tree is clean; no binaries or `build/` are committed.
[ ]	You answered all fifteen questions aloud without opening the code, or you noted the ones to revise.

CHECKPOINT — work these out before moving on
• Which three lines of the README would an interviewer most likely ask you to explain, and can you?
• Which feature would you most regret having claimed if it was only half-tested?
Hint: Open your README as a stranger and mark every claim you could not demonstrate in 30 seconds.

11. Optional extensions and the stopping rule

11.1 When you may take an extension

All of the following must be true, otherwise stop:

- Every completion criterion of Stages 0-10 is met and the README is done.
- You have at least one full planned day of buffer left (days 9-10 of the schedule).
- You pick *one* extension, time-box it to its estimate, and work on a branch (`git switch -c ext-...`).
- It ships only with tests and a README update; otherwise delete the branch. Never trade interview practice time for it.

Ext.	Feature	Value	Hours	Where the work is	Main risk
E1	`2>file`, `2>&1`, general `N>` redirection	High: extends Stage 6 understanding	2-3	Lexer (digits before `>`), `redirection` gets a target fd, `apply_redirections` uses it, `dup2(a, b)` for `2>&1`	Order of operations: `> f 2>&1` differs from `2>&1 > f`
E2	`$?` and `$VAR` expansion	Medium	3	Expansion step between lexer and parser; quoting rules decide whether it applies	Single versus double quotes; keeping tokens owned
E3	`;`, `&&`, `||` lists	Medium	3	Grammar: a list of pipelines; executor loop checks status	Parser restructuring; error cases multiply
E4	Background `&`	Medium	3-4	Parser flag; do not wait; reap with `waitpid(-1, WNOHANG)` before each prompt	Zombies, interaction with Ctrl-C, output after prompts

Hints are kept general on purpose: work out where each change belongs by tracing one example line through lexer, parser and executor, the way you did in the stages. If you cannot say which module changes, you have not understood the design yet; go back to Chapter 1.

11.2 Not planned, and why

Globbing (`*`), history and line editing (a library dependency such as readline), scripting constructs (`if`, `while`, functions), command substitution, here-documents, aliases, job control (`fg`, `bg`, process groups). Each is a large feature whose cost is out of proportion to what it adds to this project. Listing them as limitations is the right move.

11.3 The stopping rule

> **DESIGN DECISION — finished means finished**
> The project is *finished* when the final checklist (next chapter) is complete. At that point: tag `v1.0`, push, and stop adding features. A small, finished, well-understood shell is worth more than a larger one you cannot fully explain. The next step is timed practice.

11.4 Switching to timed practice

- Plan the first timed session for the day after you tag `v1.0`; do not "just add one more thing".
- Re-use what you built: dynamic arrays with doubling, string scanning with a state machine, `strtol`-style parsing, linked and nested structure cleanup, error-path discipline. Do some of your timed problems in C, because the assessment involves C.
- Apply the loop that worked here: restate the problem, trace one example by hand, write pseudocode, list edge cases, then code, then test against the edge cases. Time-box each phase.
- Keep one small notebook of mistakes (off-by-one, forgot to free, wrong loop bound) and read it before each session.

12. Final project completion checklist

Tick every box. If one stays open, that is the work that remains; nothing else is.

Functionality

[ ]	Interactive prompt (terminal only), Ctrl-D exits, Ctrl-C handled as specified
[ ]	Script on stdin works (non-interactive), including a last line without newline
[ ]	Quoting (`'..'`, `"..."`, `\x`), empty arguments, operators without spaces
[ ]	Redirection `< > >>`, including on built-ins, with errors reported
[ ]	Pipelines of any length; statuses 0/1/2/126/127/128+n as documented
[ ]	Built-ins `cd`, `pwd`, `exit [n]` with the documented error behaviour

Tests

[ ]	`make test-unit`: lexer and parser, valid and invalid input, fuzz test
[ ]	`make test-integration`: execution, built-ins, redirection, pipelines, statuses, error messages
[ ]	Descriptor-leak and zombie checks, long-line and many-pipeline stress tests
[ ]	Every fixed bug has a regression test
[ ]	Suite green in the sanitizer build and in the optimised build

Memory and resources

[ ]	No sanitizer output (ASan, LSan, UBSan) on the whole suite
[ ]	Ownership table (Stage 9) complete; every allocation has a documented owner
[ ]	Error paths audited: partial structures freed, descriptors closed, children reaped
[ ]	`ls /proc/self/fd | wc -l` prints 4 after any sequence of commands

Build and repository

[ ]	Zero warnings with `-Wall -Wextra -Wpedantic`
[ ]	Fresh clone builds and tests with the README commands only
[ ]	`.gitignore` in place; no binaries or `build/` committed; tag `v1.0`
[ ]	Small commits with meaningful messages

Documentation

[ ]	README: features, build/run, example session, tests with real numbers, architecture, decisions
[ ]	Known limitations listed honestly (no `&`, no globbing, Ctrl-Z ignored, Linux only, no `~`)
[ ]	Comments only where behaviour is non-obvious (ownership, `_exit`, `fflush`, signals)

Understanding

[ ]	I can draw the module diagram and the data flow from memory
[ ]	I can explain ownership of every structure and who frees it
[ ]	I can explain fork/exec/wait, `dup2`, `pipe` closing rules and the signal design
[ ]	I answered the fifteen interview questions aloud without the code
[ ]	I have started timed practice: the project is closed

13. References and further reading

Primary sources first. Man pages are the authority for exact behaviour; read the DESCRIPTION, RETURN VALUE and ERRORS sections. Links point to the canonical locations of each page; if one has moved, search for the page title.

Tools

- GCC manual: Warning options, Instrumentation options (sanitizers), Static analyzer options
- GNU C Library manual: Feature test macros (why `_POSIX_C_SOURCE`)
- GNU Make manual
- GDB manual
- Sanitizers: AddressSanitizer, LeakSanitizer; Valgrind manual
- Pro Git (free book)

Linux man pages (man7.org) and POSIX

Topic	Pages
Processes	fork(2), execve(2), exec(3), wait(2) / waitpid, _exit(2)
Descriptors and pipes	open(2), dup(2) / dup2, pipe(2), pipe(7)
Signals	sigaction(2), signal(7), signal-safety(7)
Library and environment	malloc(3), fgets(3), getline(3), strtol(3), chdir(2), getcwd(3), errno(3), isatty(3)
Diagnostics	strace(1), proc(5)
POSIX	Shell Command Language: quoting (2.2), redirection (2.7), exit status (2.8.2), pipelines (2.9.2), signals (2.11)

Books and courses

- OSTEP: the "Process API" chapter (cpu-api.pdf) covers fork, exec, wait and pipes with a shell as the running example.
- Michael Kerrisk, *The Linux Programming Interface* (No Starch Press): the reference for everything in Stages 4-8.
- cppreference: C: precise standard-library reference.

Appendix A. Complete public interface (all headers)

The final state of every header, for checking that your code fits together. These match the interface cards in the stages. Everything not listed here (`spawn_stage`, `child_exec`, `apply_redirections`, `run_builtin_in_parent`, `wait_for`, the built-ins, lexer and parser helpers) is `static`.

src/util.h

```
#ifndef UTIL_H
#define UTIL_H
#include <stddef.h>
void *xmalloc(size_t n);
void *xcalloc(size_t count, size_t size);
void *xrealloc(void *p, size_t n);
char *xstrdup(const char *s);
#endif
```

src/shell.h

```
#ifndef SHELL_H
#define SHELL_H
typedef struct shell_state {
    int last_status;
    int exit_requested;
} shell_state;
#endif
```

src/input.h

```
#ifndef INPUT_H
#define INPUT_H
#include <stdio.h>
typedef enum { READ_OK, READ_EOF, READ_INTERRUPTED } read_status;
read_status read_line(FILE *in, char **line_out);
#endif
```

src/lexer.h

```
#ifndef LEXER_H
#define LEXER_H
#include <stddef.h>
typedef enum { TOK_WORD, TOK_PIPE, TOK_REDIR_IN, TOK_REDIR_OUT, TOK_REDIR_APPEND } token_type;
typedef struct { token_type type; char *text; } token;
typedef struct { token *items; size_t len; size_t cap; } token_list;
void token_list_init(token_list *tl);
int  tokenize(const char *line, token_list *out, const char **err);
void token_list_free(token_list *tl);
#endif
```

src/command.h

```
#ifndef COMMAND_H
#define COMMAND_H
#include <stddef.h>
typedef enum { REDIR_IN, REDIR_OUT, REDIR_APPEND } redir_kind;
typedef struct { redir_kind kind; char *path; } redirection;
typedef struct {
    char **argv;
    size_t argc;
    redirection *redirs;
    size_t nredirs;
} command;
typedef struct { command *cmds; size_t ncmds; } pipeline;
void pipeline_free(pipeline *p);
#endif
```

src/parser.h

```
#ifndef PARSER_H
#define PARSER_H
#include "command.h"
#include "lexer.h"
int parse(const token_list *tl, pipeline *out, const char **err);
#endif
```

`src/builtins.h`

```
#ifndef BUILTINS_H
#define BUILTINS_H
#include "shell.h"
typedef int (*builtin_fn)(shell_state *sh, char *const argv[]);
builtin_fn find_builtin(const char *name);
#endif
```

`src/executor.h`

```
#ifndef EXECUTOR_H
#define EXECUTOR_H
#include "command.h"
#include "shell.h"
int execute_pipeline(shell_state *sh, const pipeline *p);
#endif
```

Dependency summary

Header	Includes	Used by
`util.h`	-	every `.c`
`shell.h`	-	main, builtins, executor
`input.h`	`stdio.h`	main
`lexer.h`	-	parser, main, tests
`command.h`	-	parser, executor, main, tests
`parser.h`	`command.h`, `lexer.h`	main, tests
`builtins.h`	`shell.h`	executor
`executor.h`	`command.h`, `shell.h`	main